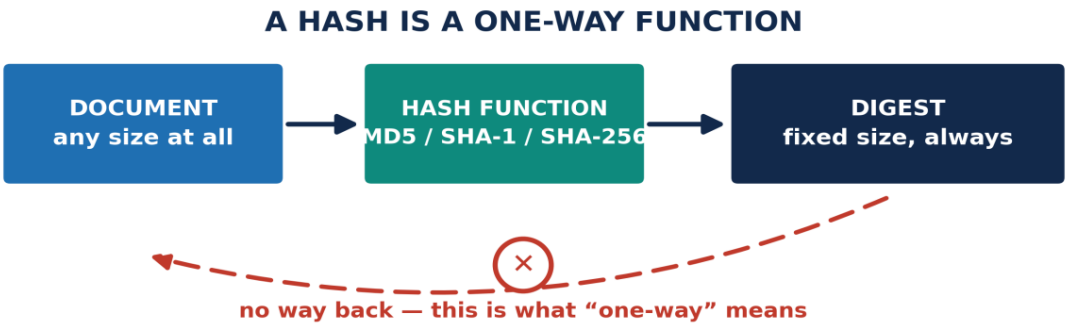
SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY)
Institute of Technical Education and Research (ITER) • Bhubaneswar, Odisha

CHAPTER 2

Hashing

A Complete, Illustrated, Line-by-Line Tutorial for B.Tech Students



Prescribed textbook
Seth James Nielson • Christopher K. Monson
Practical Cryptography in Python: Learning Correct Cryptography by Example
Apress, 2019 • Chapter 2, pages 21–52 • https://doi.org/10.1007/978-1-4842-4900-0_2

How to Use This Tutorial

As in the Chapter 1 companion, dark blue chips such as tell you exactly which printed page of Chapter 2 the material that follows corresponds to. The chips run in strict order from page 21 to page 52, so this document and the printed book stay in step throughout.

Every hash value in this document has been verified

Cryptographic tutorials are useless if the digests printed in them are wrong, so every MD5, SHA-1 and SHA-256 value that appears here was computed and checked before being included. If you type the same input into your own interpreter, you must get the same output if you do not, the mistake is in your input, not in the book.

Course learning outcomes for this chapter

1. Define a cryptographic hash function and state the two properties every hash must satisfy.
2. Use Python's hashlib to compute MD5, SHA-1 and SHA-256 digests, both in one shot and incrementally.
3. Distinguish confidentiality from integrity, and explain which one hashing provides.
4. Explain preimage resistance, second-preimage resistance and collision resistance, and give the attack cost of each.
5. Convert fluently between binary, decimal and hexadecimal, and explain why a digest is just a very large number.
6. Explain why MD5 and SHA-1 must not be used, and what "broken" actually means for a hash function.
7. Store passwords correctly using salts and a slow key-derivation function such as scrypt.
8. Mount a brute-force attack on weak password hashes and quantify how alphabet size and length change the cost.
9. Explain proof of work as used in blockchain, including the role of the nonce and the difficulty threshold.

What you need before you start

- The Python 3 virtual environment you created in Chapter 1, with the cryptography module installed.
- A terminal open beside this document. Chapter 2 is even more hands-on than Chapter 1.
- Chapter 1 completed in particular the shift cipher exercise, because Alice and Bob pick up exactly where they left off.

> **A NOTE ON TWO MISSING PAGES**
>
> The photographs supplied for this tutorial cover pages 21–33 and 36–52. Pages 34 and 35 were not included in the set. Those two pages complete the birthday-problem discussion of collision resistance, present Listing 2-5 (Avalanche) and pose Exercise 2.3.
>
> Section 12 of this tutorial supplies a clearly-labelled bridging treatment of that material so that the argument still runs continuously from page 33 to page 36. Read those two pages in the printed book as well the bridge is a scaffold, not a substitute.

BOOK PAGE 21

1. What a Hash Actually Is

Hashing is a cornerstone of cryptographic security. It involves the concept of a one-way function, or a fingerprint. Hash functions only work well when a couple of things are true about them:

- **They produce repeatable, unique values for every input.**
- **The output value provides no clues about the input that produced it.**

Some hashing functions are better at satisfying these requirements than others. This chapter deliberately covers some good ones (SHA-256) and some not-so-good ones (MD5, SHA-1) in order to demonstrate both how they work and why choosing a good one is so terribly important.

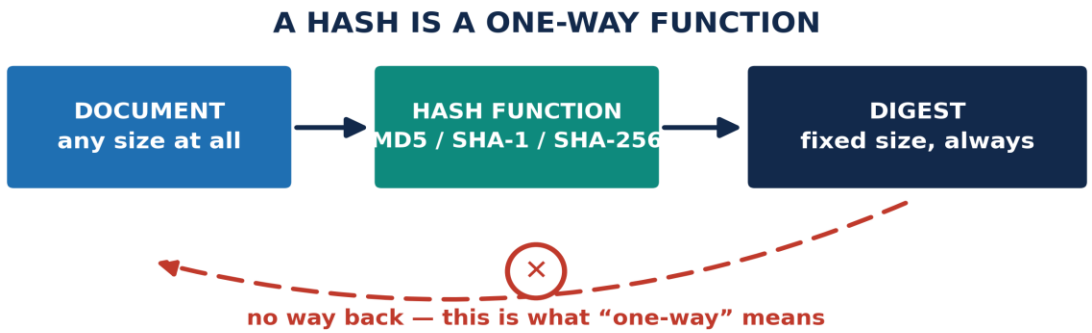


Figure 1.1 A hash squeezes an input of any size into a fixed-size digest, and the arrow only points one way. Everything in this chapter follows from those two facts.

1.1 Unpacking the two requirements

Requirement	What it means in practice	What breaks if it fails
Repeatable	The same input hashed twice, on any machine, in any year, gives the identical digest.	You could never verify a download or a password, because the answer would keep changing.
Unique	Two different inputs should not land on the same digest.	An attacker could substitute a different file that still passes your integrity check.
No clues	Seeing the digest tells you nothing whatsoever about the input.	An attacker could work backwards from a stolen password digest to the password.

REAL-LIFE CSE EXAMPLE where you already use all three

Every time git shows you a commit id like 9f2c1ae, that is a hash of the commit contents. Repeatability means everyone on your team computes the same id for the same commit. Uniqueness means two different commits cannot collide onto the same id. And the no-clues property is why the id tells you nothing about the code inside.

BOOK PAGE 21

2. Hash Liberally with hashlib

> **WARNING: MD5 IS NO GOOD**
>
> We are going to use an algorithm called MD5 for about the first half of the chapter. MD5 is deprecated and should not be used for any security-sensitive operations, or really any operations at all, except when you have to interact with legacy systems.
>
> This discussion is for introducing the concept of hashing and for providing historical context. MD5 is nice for that because it produces short hashes, has a rich history, and gives us something to break.

Read that warning carefully, because it sets the tone for the whole chapter. MD5 is used here as a teaching instrument precisely because it is weak enough to attack inside a single lab session. Nothing you learn about MD5 in this chapter licenses you to use it in a project.

2.1 Back to Alice and Bob

When we last left our two favourite spies from East Antarctica, Alice and Bob were working out some codes using simple substitution ciphers. Even though the cipher was very weak, it provided a rudimentary form of message confidentiality.

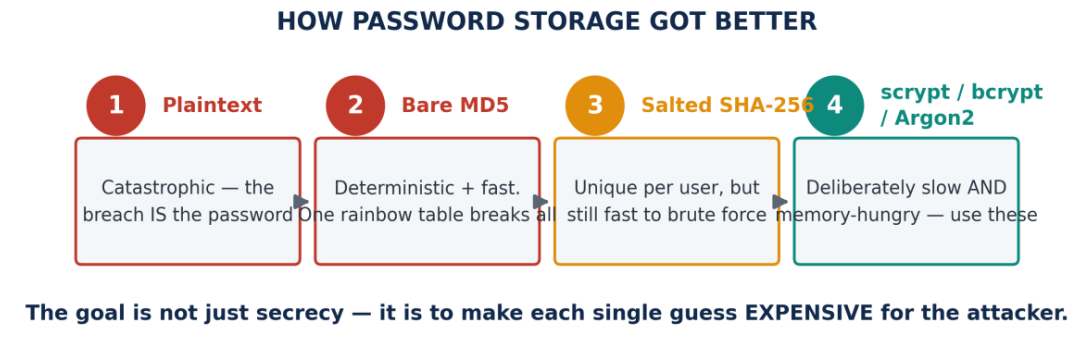


Figure 2.1 A preview of where this chapter ends up. Chapter 1 gave Alice and Bob confidentiality; hashing is about to give them integrity, and then correct password storage.

BOOK PAGE 22

2.2 Confidentiality did nothing for integrity

The substitution cipher did nothing, however, for message integrity. If you have not already guessed, message confidentiality means that nobody but the authorised parties can read the message. Message integrity means that no unauthorised parties can change the message without the change being noticed.

It is important to understand the distinction. Even with modern ciphers, just because a message cannot be read does not mean it cannot be altered, even in ways that make sense after decryption.

Also, when Alice and Bob go through customs at the WA border, sometimes their laptops are inspected. It would be nice to know that none of the files have been tampered with during that process.

	Confidentiality	Integrity
Question it answers	Can anyone else READ this?	Has anyone CHANGED this?
Chapter 1 tool	The shift cipher	Nothing at all
Chapter 2 tool		The message digest
Alice and Bob's worry	WACKO reading intercepted traffic	A customs officer editing files on a laptop

2.3 Enter the message digest

Fortunately for Alice and Bob, their new technology officer introduces them to something called a "message digest", used to "fingerprint" their files and message transmissions. He explains that they can combine their messages' contents with message digests, then use the two together to tell whether part of any message was altered. That sounds like just the thing.

Since they do not know anything about digests, it is time for some training. Let's follow along with their instructor in our own Python interpreter, starting with Listing 2-1.

2.4 Listing 2-1 Intro to hashlib

Listing 2-1. Intro to hashlib

```
>>> import hashlib
>>> md5hasher = hashlib.md5()
>>> md5hasher.hexdigest()
'd41d8cd98f00b204e9800998ecf8427e'
```

Line by line

Line(s)	What that line actually does
1	Imports Python's standard-library hashing module. It is built in no pip install is required. It provides MD5, SHA-1, the whole SHA-2 family and more, all behind one uniform interface.
2	Creates a **hasher object**, not a hash. Think of it as an empty container that you can keep feeding data into. Because we passed no argument, the container is currently empty.

Line(s)	What that line actually does
3	Asks the object for its digest so far, rendered as hexadecimal text.
4	The result. This is the MD5 digest of the **empty document** a famous constant you will meet again and again in security work. Notice we still got a full-length digest even though we hashed nothing at all: the output size never changes.

Importing a library called hashlib seems straightforward enough, but what is md5? The instructor explains that the “MD” in MD5 stands for “message digest”. We will get into some interesting details in just a moment, but for now, a digest like MD5 converts a document of any length (even an empty document) into a large number that takes up a fixed amount of space. It should have at least these features:

- The same document always produces the same digest.
- The digest “feels” random: if you have a digest, it gives you no clues about the document.

BOOK PAGE 23

2.5 The fingerprint analogy

In this way, a digest is like a fingerprint and is sometimes called one: it is a small amount of data that stands in for the document's identity. Every document we might ever care about should have a completely unique digest.

Human fingerprints are similar in other ways. If you have a person at hand, it is easy to produce a relatively consistent and unique fingerprint; but if the only thing you have is a fingerprint, it is not so easy to find out whose it is. Digests work the same way: given a document, it is easy to calculate its digest; but given only a digest, it is very hard to find out what document produced it. Very hard. The harder, the better, in fact.

Fingerprint	Message digest
Small compared with the person	16 or 32 bytes compared with a whole file
Easy to take, given the person	Easy to compute, given the document
Very hard to identify the person from the print alone	Very hard to recover the document from the digest alone
Two people should not share one print	Two documents should not share one digest

2.6 Why the empty digest was still full length

The MD5 digest creates a number that always occupies 16 bytes of memory. In our example interpreter session, we asked it to produce a digest for the empty document, which is why we did not add any data to the md5hasher before asking it to produce a digest for us.

The use of hexdigest is shown to demonstrate a more human-readable format for the number, where each of the 16 bytes in the digest is shown as a two-character hexadecimal value.

16 bytes × 2 hex characters per byte = **32 characters**. Count the characters in `d41d8cd98f00b204e9800998ecf8427e` and you will find exactly 32. Every MD5 digest you will ever see has this length, no matter what went in.

2.7 Listing 2-2 Hash Names

The instructor, anxious to move on, asks Alice and Bob to hash each of their names (expressed as bytes). To the interpreter, and Listing 2-2!

Listing 2-2. Hash Names

```
>>> md5hasher = hashlib.md5(b'alice')
>>> md5hasher.hexdigest()
'6384e2b2184bcbf58eccf10ca7a6563c'
>>> md5hasher = hashlib.md5(b'bob')
>>> md5hasher.hexdigest()
'9f9d51bc70ef21ca5c14f307980a29d8'
```

Line(s)	What that line actually does

1	This time we pass data straight into the constructor. The hasher is created and fed in a single step.
1	Note the **b** prefix on `b'alice'`. That marks a **byte string**, not a text string. Hash functions consume bytes, and Python 3 will refuse a plain text string with a TypeError. This is explained fully on the next page.
2 – 3	Produce and display the digest for `alice`.
4 – 6	The same again for `bob`. Notice we *rebound* the name `md5hasher` to a brand new object. We did not add “bob” to the existing hasher that would have given us the digest of “alicebob” instead.

2.8 Listing 2-3 Combine Operations

For short strings like these, it is not uncommon to combine operations, as in Listing 2-3.

Listing 2-3. Combine Operations

```
>>> hashlib.md5(b'alice').hexdigest()
'6384e2b2184bcbf58eccf10ca7a6563c'
>>> hashlib.md5(b'bob').hexdigest()
'9f9d51bc70ef21ca5c14f307980a29d8'
```

Both calls are chained: construct the hasher, feed it the data, and ask for the hex digest, all in one expression. The results are identical to Listing 2-2, which is the point this is purely a convenience of style, not a different operation.

“So, Alice, Bob, what did you learn from this?” the instructor asks. When neither one answers, she suggests that they experiment some more. Let’s follow along.

BOOK PAGE 24

3. Bytes, Not Strings

Python differentiates between Unicode strings and raw byte strings. A full explanation of the differences is beyond the scope of this book, but for almost all cryptographic purposes, you must use bytes. Otherwise you can end up with some very nasty surprises when the interpreter attempts (or refuses) to convert Unicode strings into bytes for you.

We forced our string literals to be bytes using the `b''` string syntax. In other examples where user input requires us to start with Unicode strings, we will encode those to bytes ensuring that it is safe to do so.

See the failure for yourself

what happens if you forget the b prefix

```
>>> import hashlib
>>> hashlib.md5('alice')
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: Unicode-objects must be encoded before hashing

>>> hashlib.md5('alice'.encode('utf-8')).hexdigest()
'6384e2b2184bcbf58eccf10ca7a6563c'
```

WHY THIS RULE EXISTS

A hash is a mathematical operation on numbers, and bytes are numbers. Text is not text is an abstract sequence of characters that must be encoded into bytes before any arithmetic can touch it, and the same text encodes to different bytes under different encodings.

If Python silently guessed an encoding for you, the identical string could hash to different digests on two machines with different locale settings. Refusing to guess is the safe behaviour, and the TypeError above is Python protecting you.

EXERCISE 2.1 WELCOME TO MD5

Compute more digests. Try computing the MD5 sum of the following inputs:

• b'alice' (again) • b'bob' (again) • b'balice' • b'cob'

• b'a' • b'aa' • b'aaaaaaaaaa' (ten copies of the letter “a”)

• b'a'*100000 (100,000 copies of the letter “a”)

What did you learn about MD5 sums from Exercise 2.1?

Worked answers check your own output against these

Input	MD5 digest
b'alice'	6384e2b2184bcbf58eccf10ca7a6563c
b'bob'	9f9d51bc70ef21ca5c14f307980a29d8
b'balice'	6760742ebf884c998752b4e082b78224
b'cob'	386685f06beecb9f35db2e22da429ec9
b'a'	0cc175b9c0f1b6a831c399e269772661
b'aa'	4124bc0a9335c27f086f24ba207a4912
b'aaaaaaaaaa'	e09c80c42fda55f9d992e59ca6b3307d
b'a'*100000	1af6d6f2f682f76f80e606aeaaee1680

The three things you were meant to learn

1. The same input always gives the same digest. Hashing b'alice' a second time reproduces the first answer exactly. Hash functions are deterministic.
2. A tiny change to the input completely destroys any resemblance in the output. b'alice' and b'balice' differ by one letter, yet their digests share nothing. This is called the avalanche effect.
3. The digest length never changes. One byte in or 100,000 bytes in, out comes exactly 32 hex characters. Hashing compresses, and it compresses to a fixed size.

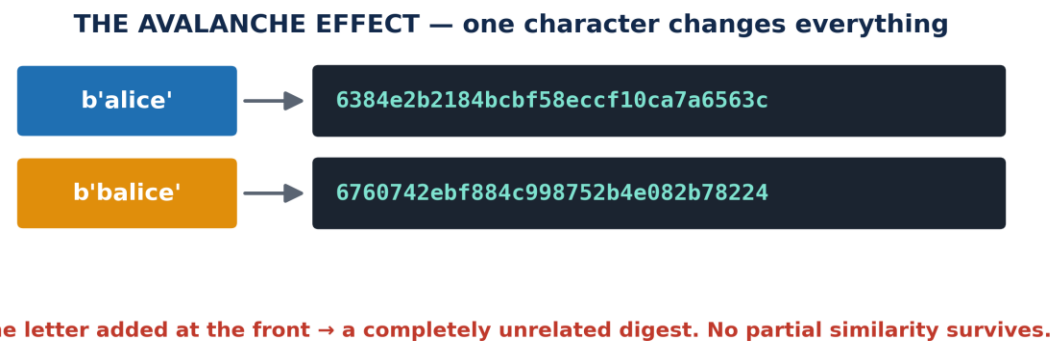


Figure 3.1 The avalanche effect, using two of the digests you just computed. Adding a single letter at the front changes every part of the output.

BOOK PAGE 25

4. Listing 2-4 Hash Update

So far we have handed the whole message to the hasher at once. But a digest does not require the entire input all at once. It can be inserted a chunk at a time using the update method, as shown in Listing 2-4.

Listing 2-4. Hash Update

```
>>> md5hasher = hashlib.md5()
>>> md5hasher.update(b'a')
>>> md5hasher.update(b'l')
>>> md5hasher.update(b'i')
>>> md5hasher.update(b'c')
>>> md5hasher.update(b'e')
```

4.1 Line by line

Line(s)	What that line actually does
1	Creates an *empty* hasher no data passed to the constructor this time. The object now holds the internal state of a hash computation that has consumed nothing.
2	Feeds one byte into that internal state. Note that `update()` returns nothing it mutates the object in place. Beginners often write `h = h.update(b'a')` and end up with `None`.
3 – 6	Four more single-byte updates. The hasher is now in exactly the state it would have been in had we written `hashlib.md5(b'alice')` because concatenating the chunks reproduces the original message.

4.2 The instructor's question

The instructor asks Alice and Bob, "What do you think the output of the md5hasher.hexdigest() instruction will be?" Try it out and see if you got it right!

the answer

```
>>> md5hasher.hexdigest()
'6384e2b2184bcbf58eccf10ca7a6563c'
```

It is the digest of "alice" exactly the value from Listing 2-2. Feeding five one-byte chunks is indistinguishable from feeding one five-byte message, because the hash function only ever sees a stream of bytes and has no notion of where your chunk boundaries were.

> **REAL-LIFE CSE EXAMPLE why update() exists at all**
>
> You cannot load a 4 GB Ubuntu ISO into a Python variable to hash it your machine would run out of memory. Instead you open the file, read it in 64 KB blocks, and call update() on each block in turn. The hasher keeps only a few dozen bytes of internal state no matter how big the file is.
>
> The same pattern is behind the progress bar you see when a download manager verifies a checksum, and behind streaming hash computation in git, rsync and every backup tool you have ever used.

“Great,” the instructor says when they have finished. “Your introductory training is almost over. Just one more exercise!”

EXERCISE 2.2 GOOGLE KNOWS!

Do a quick Google search using the following hashes (enter the hashes literally into the Google search bar):

1. 5f4dcc3b5aa765d61d8327deb882cf99
2. d41d8cd98f00b204e9800998ecf8427e
3. 6384e2b2184bcbf58eccf10ca7a6563c

What you will find, and why it matters

Hash	What it is the MD5 of	The lesson
5f4dcc3b…	the word “password”	The single most common password on earth. It will reappear in Section 15 of this tutorial.
d41d8cd9…	the empty input	You computed this yourself in Listing 2-1.
6384e2b2…	b'alice'	Even a short, ordinary word is already indexed and searchable.

THIS IS THE WHOLE POINT OF THE EXERCISE

MD5 is a one-way function but that guarantees nothing if somebody has already published the answer. Because hashing is deterministic, anyone can pre-compute the digests of every common word and put them in a lookup table. These are called rainbow tables, and Google itself functions as an informal one.

The mathematics was never broken here. The attacker simply did the work in advance. Keep this in mind: it is the exact weakness that salting (Section 16) exists to remove.

BOOK PAGE 25 – 26

5. Making a Hash of Education

Within the realm of computer security, the terms “hashing” or “hash function” always refer to cryptographic hash functions, unless otherwise stated. There are some very useful non-cryptographic hash functions as well. In fact, you were taught a very simple one in grade school: computing whether a number is even or odd. Let’s see how this simple, familiar function illustrates principles that apply to all hash functions.

5.1 Domain and range

Hash functions are fundamentally trying to map an enormous (even infinite) number of things onto a (relatively) small set of things. When using MD5, for example, no matter how big our document is, we end up with a 16-byte number. In discrete algebra terms, this means that the domain of a hash function is much larger than its range. Given a very, very large number of documents, chances are that many of them will produce the same hash.

5.2 Hash functions are lossy

Hash functions are therefore lossy. We lose information going from our source document to a digest or hash. This is actually critical to their function, because without a loss of information, there would be a way to go backward from the hash to the document. We really do not want that, and we will see why soon.

Thus, computing whether a number is even or odd fits this description quite well. No matter how large or interesting the (integer) number, we can squash it down into a single bit of space: 1 for odd, 0 for even. That is a hash! Given any number of any size, we can efficiently produce its “oddness” value, but given its oddness, we would be hard-pressed to figure out which number produced it. We can create a very, very large number of possible inputs, but we cannot know which one specifically was used to make that answer.

> **NOTE the parity bit**
>
> An “even or odd” bit is sometimes called a “parity” bit and has often been used as a rudimentary error detection code.

5.3 The three qualities every hash has

The even/odd hash example illustrates this principle of “squashing” an input down to a fixed size value. This value is consistent, meaning you will not get a different value out if you put the same number into it twice. It compresses large inputs into a fixed-size space (just one bit!), and it is lossy: you cannot tell which number was used as input by examining only the output.

Quality	Definition	In the even/odd hash	In MD5
Consistency	Same input, same output, every time	7 is always odd	b'alice' always gives 6384e2b2…

Quality	Definition	In the even/odd hash	In MD5
Compression	Large input space → small fixed output	Any integer → 1 bit	Any file → 16 bytes
Lossiness	Information is destroyed on the way	“odd” could be 1, 3, 5, 7 …	Countless documents share a digest

5.4 What makes a hash CRYPTOGRAPHIC

All hash functions, including non-cryptographic hash functions, have the fundamental qualities of consistency, compression, and lossiness and have all kinds of important applications in computer science. These qualities alone, however, are not enough for a hash function to be cryptographic or secure: for those, a hash function needs a few more properties:

- **Preimage resistance**
- **Second-preimage resistance**
- **Collision resistance**

We will talk about each of these important qualities in turn.

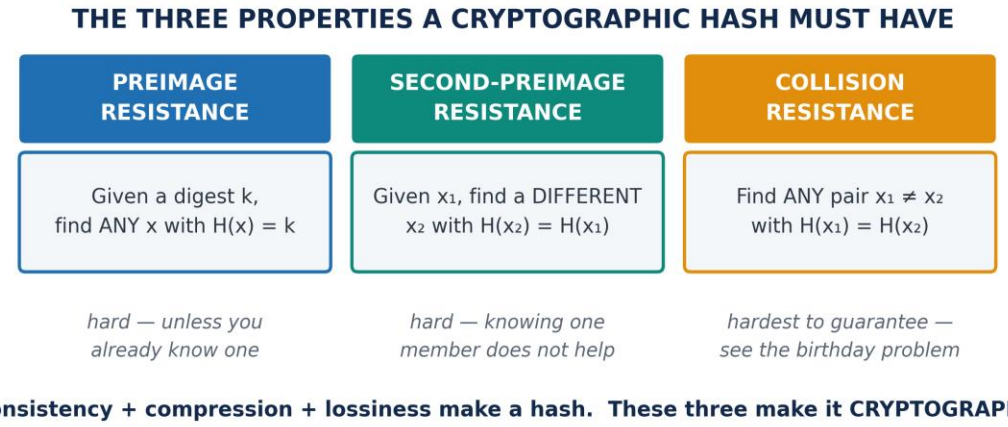


Figure 5.1 The three properties that separate a cryptographic hash from an ordinary one. The even/odd hash has none of them.

BOOK PAGE 27

6. Preimage Resistance

Informally, a preimage is the set of inputs for a hash function that produce a specific output. If we were to apply that to our parity example from earlier, the preimage for an odd parity bit is the (infinite!) set of all odd integers. Similarly, the preimage for an even parity bit is the set of all even integers.

6.1 The definition, stated precisely

What does this mean for a cryptographic hash? Earlier, we computed that the MD5 hash value `6384e2b2184bcbf58eccf10ca7a6563c` could be generated by the input `b'alice'`. Thus, the preimage of

```
MD5(x) = 6384e2b2184bcbf58eccf10ca7a6563c
```

contains the element `x == b'alice'`.

This is important, so let's state it in more precise terms (using integers in our domain and range remember, a document is ordered bits and is therefore just a big integer):

> **DEFINITION PREIMAGE**
>
> A preimage for a hash function H and a hash value k is the set of values of x for which H(x) = k.

For cryptographic hash functions, this concept of the preimage is important. If I give you a digest value, there might (should) be infinitely many input numbers that could be used to produce it. Those numbers are the preimage for that digest. Remember, every document is just a large integer number from the computer's point of view. It is all just bytes, and we are just performing a mathematical operation on them. The preimage is therefore just an infinite set of integer numbers.

6.2 Preimage RESISTANCE

The idea of preimage resistance is basically this: if you hand me a digest and I do not already know how you got it, I cannot even find one element in the preimage for it without doing a ridiculous amount of work. Ideally I would have to do an impossible amount of work.

It is already hard to (in general) find the entire preimage; it is way too big. What we are really interested in is making it tough to find any element in the preimage unless you already happen to know one. That is where lossiness comes in: the digest should give us no information whatsoever about the document that produced it.

> **FOOTNOTE 1 what a good preimage looks like**
>
> If thinking about the domain helps, a good quality for every preimage of a hash function would be that all of its elements are very spread out with unpredictable spacing. That way you are very unlikely to accidentally choose one by guessing (they are really spread out), and given a hit, you are just as unlikely

to find any others (unpredictable spacing). That last part is something we will dive into in just a moment.

BOOK PAGE 28

6.3 Inverting the hash

Without any information to guide us, the best we can do is random guessing or trying everything until we accidentally land on one that produces the right digest. That is preimage resistance.

The process of attempting to find an element in the preimage for a given output is called inverting the hash: trying to run it backward to get an input for a given output. Preimage resistance means that finding any inverse is hard.

This is why the even/odd function is a potentially useful hash function, but not a secure hash function. If I give you an even/odd value, for example, you can readily come up with something that matches. I say "even", you say "2", for example. That is not very preimage-resistant, because you just told me an input that produces the given output, and you did not have to think very hard to do it. In fact, you can describe the entire preimage without much trouble: "all even integers".

6.4 Why MD5 is hard to invert but not impossible to break

For cryptographic hash functions, if I tell you MD5(x) = `ca8a0fb20578205 1bd49f02eae17c9ee`, you (ideally) cannot tell me what x is unless you can find someone who already knows and is willing to tell you. MD5 is hard to invert.

Now, you could just try random (or ordered) documents to see if any of them produce `ca8a0fb202057820 51bd49f02eae17c9ee`, and you might get (very!) lucky. That approach is one kind of a **brute-force attack**, because you just have to pick your way through every single straw in the haystack to find the needle you are looking for. All you can do is commit to looking at an awful lot of straw, relying on raw stamina to carry you through.

6.5 Consistency is what makes lookup attacks possible

Because consistency is a property of hashes, if you have already found an input that maps to a given output, you can find it by searching Google, for example, and then that particular output is trivially inverted.

The ASCII text "alice" always maps to **`6384e2b2184bcbf58eccf10ca7a6563c`** when run through MD5, no matter what, so if you happen to know that those two things go together, you can easily find "alice" from the digest.

For that particular output, though, MD5 is trivially inverted. That does not mean MD5 is not preimage-resistant: to break that you would need to be able to find an output always without knowing one beforehand.

> **THE DISTINCTION THAT WILL BE ON YOUR EXAM**
>
> Inverting ONE known digest by looking it up is not breaking preimage resistance. Breaking preimage resistance means having a general method that inverts an arbitrary digest one you have never seen before and nobody has published substantially faster than brute force.

> Students routinely lose marks by writing “MD5 is not preimage-resistant because you can Google the hash of password”. That is a statement about rainbow tables, not about the mathematics of MD5.

6.6 So how long would brute force take?

That leaves us with brute force again. How long would it take to “guess” an element of the preimage for an MD5 hash using a brute-force technique (either random guessing or sequential searching)? To answer this, we first need to look at how many possible hash values there are.

We know that MD5 always produces a 16-byte digest, and we can use that to figure out how hard it should ideally be to invert MD5; to do so, we will need some understanding of binary (base-2), decimal (base-10), and hexadecimal (base-16) positive integers (plus 0, but we usually just say “non-negative”).

If you already understand those pretty well, feel free to skip to the next section.

BOOK PAGE 29

7. Byte into Some Non-negative Integers

Most computers use binary to represent everything. The binary number system is represented in base 2. One nice way to be introduced to it is through counting. Here we have the familiar base-10 (decimal) number on the left and the corresponding binary number on the right:

Decimal	0	1	2	3	4	5	6	7	8	9
Binary	0	1	10	11	100	101	110	111	1000	1001

7.1 How counting works in this system

Adding 1 gives us 1, which is expected. So far, so good. But then, since we are in base 2, we run out of digits when we try to do it again! Just like there is no single digit representing the number “10” in our decimal system, there is no single digit representing “2” in binary.

What do we do when we run out of digits in base 10? We use place values. The very number “10” shows this: there is “1 ten” and “0 ones” in that number. It is the number that comes after “9”.

Binary is similar. When we move up one number from “1”, we run out of digits, so we put a “1” in the “twos column” and start over at “0” in the “ones column”.

7.2 Bases in general

What might seem remarkable is the fact that you can represent every non-negative integer this way, just like you can with decimal. The base value (“base-2”, “base-10”, “base-16”, etc.) tells you how many digits you have to work with and therefore what the place values mean.

Here are a few place values in different number systems. Note that people get a little bit careless with these things, using decimal to talk about them, but really the number system is arbitrary. When it comes to that, there are ten kinds of people in the world: those who understand binary and those who don’t.

FOOTNOTE 2

An old joke. You’re welcome. And we’re sorry.

IF THE JOKE DID NOT LAND

“Ten kinds of people” is written 10 in binary, which is two. So there are two kinds of people. If you had to have that explained, you are now firmly in the second group but not for much longer.

BOOK PAGE 30

7.3 What does "10" mean?

That is a big problem when teaching number systems: what does "10" mean without knowing what base we are operating in? The default is to assume it means "ten" unless the base is explicitly stated, or is really obvious, like with hexadecimal, where we see "a"–"f" along with the more common decimal digits. We will do that here too: if you do not see a base or you cannot easily tell what it is, you are looking at decimal.

7.4 Place values, two ways

	Place 3	Place 2	Place 1	Place 0
Binary	8	4	2	1
Decimal	1000	100	10	1
Hexadecimal	4096	256	16	1

Or, put another way:

	Place 3	Place 2	Place 1	Place 0
Binary	2^3	2^2	2^1	2^0
Decimal	10^3	10^2	10^1	10^0
Hexadecimal	16^3	16^2	16^1	16^0

All of these number systems work in the same way: place value is determined by adding one to an exponent on the base.

7.5 A worked conversion

In decimal, therefore, the number 237 really means $2 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0 = 200 + 30 + 7$.

The same number in hex (we will use x_h to mean "x in hexadecimal") is ed_h, which means $e_h \cdot 10_{1h} + d_h \cdot 10_{0h}$. But what does that mean? Well, $e_h = 14_d$ in decimal, and $d_h = 13_d$. Since 10_h has a 1 in the sixteens column, we get (in decimal) $14 \cdot 16 + 13 = 237$.

Base	Written as	Working	Value
Decimal (10)	237	2·100 + 3·10 + 7·1	237
Hexadecimal (16)	ed	14·16 + 13·1	237
Binary (2)	11101101	128+64+32+8+4+1	237

7.6 Why hexadecimal at all?

Why do we care about hexadecimal in the first place, other than its relative compactness? Hexadecimal (or "hex") is useful because its place values are multiples of 2 they are multiples of 2^4, to be exact so it lines up nicely with binary.

BOOK PAGE 31

Consider the following table with hex on the left and binary on the right:

Hex	0	1	2	3	4	5	6	7
Binary	0	1	10	11	100	101	110	111

Hex	8	9	A	B	C	D	E	F
Binary	1000	1001	1010	1011	1100	1101	1110	1111

We ran out of digits in hex at exactly the same time that we needed to go from four columns to five in binary! That is really helpful, because it means we can trivially convert back and forth between a computer's native and sprawling binary numbers to the much more human-friendly and compact hex numbers. People even get good enough at this that they can just translate them on sight.

Here is an example with binary on top and hex underneath:

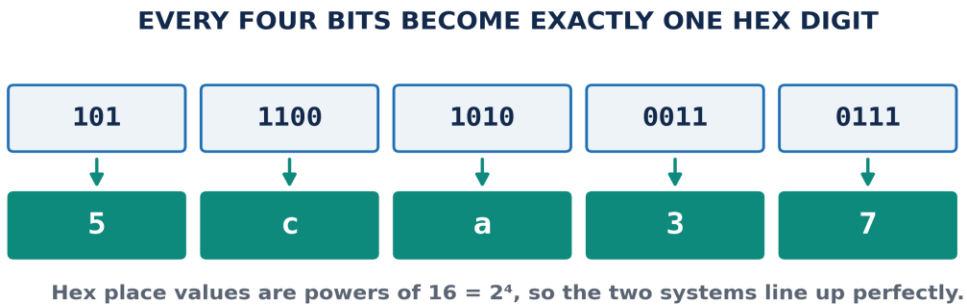


Figure 7.1 Group the bits into fours from the right, and each group becomes one hexadecimal digit. Nothing is lost and nothing is approximated.

No matter how big a binary number gets, you can take every four bits and write them as a single hexadecimal digit.

7.7 The point of the detour

The point of walking through this review of binary is to emphasise once again that every sequence of bits in a computer is a number. What if those bits are a document? That is a number. What about if they represent an image? That is just a big number.

> **REAL-LIFE CSE EXAMPLE why you keep meeting hex**
>
> Every MAC address, every IPv6 address, every colour in CSS (#1F6FB2), every memory address in a debugger and every byte you inspect in Wireshark is printed in hex all for the same reason: four bits per character, so the mapping to the underlying bits is exact and reversible by eye.

BOOK PAGE 32

THE MEANING IS NOT IN THE COMPUTER

The “meaning” of those bits is not in the computer, it’s in our minds.

We may display the bits a certain way, but we humans choose to do that based on what we think they mean. The computer has no idea what they really mean. They’re just numbers. Can we store the meaning itself somehow? Well, sure, but that would force us to encode the meaning as a number, because numbers are all that computers understand. Even their instructions are just numbers.

Philosophize much, do we? It’s actually a pretty important thing to understand if you really want to know how computers work, and we definitely need people in the world who do. Data and code are just big numbers, and computers basically just fetch, store, and do arithmetic on them.

8. How Hard a Hash!

With that little side trip, we can now answer what we wanted to answer in the first place: how hard would it be, in general, to invert MD5 using brute force? We can take a stab at this by looking at the size of its output.

MD5 outputs a value in 16 bytes, which is $16 \cdot 8 = 128$ bits. With n bits we can express 2^n individual values, so MD5 can output a lot of different digests. This many, in fact (in decimal):

340,282,366,920,938,463,463,374,607,431,768,211,456

8.1 Putting that number in human terms

Even if you checked 1 million values per second (and were guaranteed that nothing you checked produced an output you had seen before), it would still take you about 10^{26} years (100 million billion billion!) to find a suitable input by brute force.

By comparison, our sun is only expected to continue sustaining earth life for at most another 5 billion years; your computer would need to keep running for many, many times that long.

Quantity	Value
MD5 output size	16 bytes = 128 bits
Number of possible digests	$2^{128} \approx 3.4 \times 10^{38}$
Rate assumed for the estimate	1,000,000 hashes per second
Time to exhaust the space	$\approx 10^{26}$ years
Remaining lifetime of the sun (for life on earth)	$\approx 5 \times 10^{9}$ years
Ratio	the attack outlasts the sun by a factor of about 10^{16}

8.2 The honest caveat

If you have a cryptographic algorithm whose only means of being broken is brute force, you have a good algorithm. The trouble is, you do not necessarily know that it is good. But this gives us an upper bound on

how long it would take to find an input that produces a particular hash in MD5. At least it will not take longer!

FOOTNOTE 3

In hex, this number is much more tightly related to binary and looks a little more sensible: 10000000000000000000000000000000.

READ SECTION 8.2 TWICE

This is the single most important sentence in the whole chapter, and the rest of Chapter 2 is a demonstration of it. The 10^{26}-year figure is the cost of the DUMBEST possible attack. It is an upper bound, not a guarantee.

Every real break of a hash function comes from finding a shortcut that beats brute force. Sections 13 and 14 show exactly that happening to MD5 and SHA-1, and the numbers collapse from 10^{26} years to under an hour.

BOOK PAGE 33

9. Second-Preimage and Collision Resistance

Once preimage resistance is understood, the other two properties are relatively easy to grasp. We wandered a bit into brute force and binary near the end of that last section, so let's quickly review:

> **PREIMAGE RESISTANCE recap**
>
> Preimage resistance means it is really hard to find a document that produces a particular digest, unless you already know one.

9.1 Second-Preimage Resistance

Second-preimage resistance means that if you already have one document that produces a particular digest, it is still really hard to find a different document that produces that same digest.

In other words, just because you know that

```
MD5(alice) = 6384e2b2184bcbf58eccf10ca7a6563c
```

it doesn't mean you can find another value to put into MD5(·) that will give you the same value. You would have to resort to brute force again.

To tie it back to its name, if you have one member of the preimage already, it is not any easier to find a second member of the preimage: there is no exploitable pattern in the preimage.

9.2 Collision Resistance

Collision resistance is a bit more subtle than either of the preimage characteristics we just mentioned. Collision resistance means that it is hard to find any two inputs that produce the same output: not a specific output, just the same output.

9.3 The birthday analogy

A classic way of describing this is by using birthdays. Suppose you are in a room full of people and you want to find two of them whose birthday is February 3. How likely is that? Not necessarily very likely, if you really picked it at random.

But now let's say you want to do something else. You want to know whether any two people have the same birthday. You don't care what day of the year it falls on, you just want to know whether anyone's birthday overlaps with anyone else's. How likely is that?

> **FOOTNOTE 4**
>
> The "birthday problem" is a classic problem in probability theory, of uncertain origin.

The question you ask	Hash equivalent	How hard

Is anyone born on 3 February?	Preimage / second-preimage: hit ONE specific target	Very hard roughly 2^n work
Do ANY two people share a birthday?	Collision: any matching pair will do	Far easier roughly $2^{n/2}$ work

That difference is the whole reason collision resistance is the first property of a hash function to fall, and it is exactly what happened to MD5.

BOOK PAGES 34 – 35 BRIDGING SECTION

10. Bridge The Birthday Bound, Avalanche and Exercise 2.3

PLEASE READ THIS FIRST

Pages 34 and 35 were not among the photographs supplied for this tutorial. This section is written from the surrounding context page 33 sets up the birthday problem and page 36 resumes mid-argument with MD5 collision numbers so that the reasoning still runs without a gap.

Treat this section as a bridge, not as the book's own words. Read those two printed pages, which contain the completion of the birthday argument, Listing 2-5 (Avalanche) and Exercise 2.3, and update your notes accordingly.

10.1 Completing the birthday argument

The answer to the birthday question surprises almost everybody. In a room of only 23 people, there is already a better-than-even chance that two of them share a birthday. With 70 people the probability is over 99.9%. The reason is that you are not comparing each person against one fixed date; you are comparing every person against every other person, and the number of pairs grows as the square of the number of people.

Applied to hash functions, this gives what is called the birthday bound. If a hash produces n bits of output, then finding a preimage takes about 2^n attempts, but finding a collision takes only about 2 raised to the power n/2. Squaring the difficulty of one problem halves the exponent of the other.

Hash	Output bits	Preimage attack (ideal)	Collision attack (birthday bound)
MD5	128	2^{128}	2^{64}
SHA-1	160	2^{160}	2^{80}
SHA-256	256	2^{256}	2^{128}

This table is worth memorising, because it explains why the chapter keeps quoting the number 2^{64} for MD5 collisions when the digest is 128 bits long. It also explains why SHA-256 is considered comfortable today: its collision resistance alone is as strong as MD5's entire preimage resistance was supposed to be.

10.2 The avalanche effect (Listing 2-5)

The other idea developed on those pages is the avalanche effect, which you already observed empirically in Exercise 2.1. A good cryptographic hash is built so that flipping a single bit of the input flips, on average, half of the bits of the output and which half is unpredictable.

You can measure this yourself. The following program is written in the spirit of the book's Listing 2-5 and reproduces the same lesson.

An avalanche measurement you can run now

```
1  import hashlib
2
3
4  def bits(digest_bytes):
5    return "".join(format(b, '08b') for b in digest_bytes)
6
7
8  def avalanche(a, b):
9    ha = bits(hashlib.md5(a).digest())
10   hb = bits(hashlib.md5(b).digest())
11   differing = sum(1 for x, y in zip(ha, hb) if x != y)
12   print("{} vs {}".format(a, b))
13   print(" {} of {} bits differ ({:.1f}%)".format(
14     differing, len(ha), 100 * differing / len(ha)))
15
16
17 if __name__ == "__main__":
18   avalanche(b'alice', b'balice')
19   avalanche(b'alice', b'alicf')
```

Line(s)	What that line actually does
4 – 5	Converts the raw digest bytes into a string of 128 binary characters. Note the use of `.digest()` rather than `.hexdigest()` we want the actual bytes, not their hex rendering.
8 – 10	Hashes both inputs and converts each digest to its bit string.
11	`zip` walks the two bit strings in parallel, and the generator counts every position where they disagree. This count is called the Hamming distance.
12 – 14	Reports the count and the percentage. For a well-designed hash the answer will hover around 50%, whatever pair of similar inputs you choose.
19 – 20	Two test pairs: one differing at the front, one differing at the back. Position makes no difference that is the point.

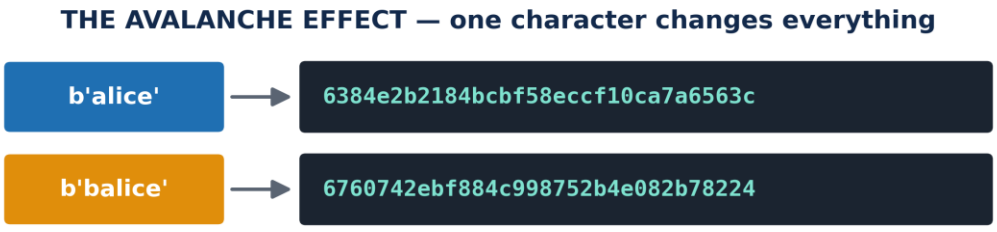


Figure 10.1 The avalanche effect made visible. Around half of all 128 output bits flip when one input character changes.

EXERCISE 2.3 (reconstructed from context)

Exercise 2.3 falls on these missing pages. Based on the surrounding material, the following is a faithful substitute confirm the exact wording against your printed copy:

Run the avalanche measurement above on several pairs of nearly identical inputs. Roughly what fraction of the output bits change each time? Does it depend on where in the input you make the change, or on how long the input is? What would it tell you about a hash function if the answer were consistently much less than half?

BOOK PAGE 36

11. What “Broken” Means for MD5

Tries with half as many zeros in it: the difference is astounding. Concretely for MD5, finding a document for a given digest should take about 2^{128} attempts, where finding two documents that collide should only take 2^{64} attempts.

As it happens, MD5’s collision resistance is actually nowhere near that good. It has been “broken”, meaning that techniques have been discovered for finding collisions in far fewer than the expected 2^{64} attempts. In short, the problem can be solved by something other than brute force and in less than an hour. Keep that in mind, and we will come back to it later on.

Attack on MD5	Ideal cost	Actual cost today	Status
Find a preimage	2^{128}	≈ 2^{123} (see Section 13)	Weakened, not practical
Find a second preimage	2^{128}	no practical attack	Weakened, not practical
Find a collision	2^{64}	seconds to under an hour	Thoroughly broken

12. Digestible Hash Hashing a File

By this point, you should know enough to create a Python program that computes the MD5 digest of a file. This is a common use for hashes and a good exercise to work through.

12.1 The three things that trip students up

1. Remember, you must use Python bytes, not Python Unicode strings, as inputs.
2. If you try to open a Python file with the default mode, it may open it as a text file and read the data as strings, doing implicit decoding. You should, instead, open the file in “rb” mode so that all reads produce raw bytes.
3. For a text file, you might be tempted to read the data as a string and then use the string’s encode method to convert to bytes, but depending on configuration, this encoding may not be what you expect and lead to nasty surprises.

A correct file hasher note the "rb" and the chunked read

```
1  import hashlib
2  import sys
3
4
5  def md5_of_file(path, block_size=65536):
6    hasher = hashlib.md5()
7    with open(path, "rb") as f:
8      while True:
```

```
9         block = f.read(block_size)
10        if not block:
11          break
12        hasher.update(block)
13    return hasher.hexdigest()
14
15
16  if __name__ == "__main__":
17    print(md5_of_file(sys.argv[1]))
```

Line(s)	What that line actually does
5	A block size of 64 KB is a sensible default. It is large enough that we are not making millions of tiny reads, and small enough that memory use stays flat no matter how big the file is.
7	The critical line. **"rb"** means read, binary. Open the same file in the default "r" mode and Python decodes it as text, mangles line endings on some platforms, and gives you a completely different digest or a UnicodeDecodeError on a binary file.
8 – 12	The streaming loop. `f.read()` returns an empty bytes object at end of file, which is falsy, so that is our exit condition. Each block is fed to the hasher with `update()` exactly the technique of Listing 2-4.
13	Returns the hex digest once the whole file has been consumed.
17	Takes the file path from the command line so the script behaves like the `md5sum` utility you are about to meet.

EXERCISE 2.4 MD5 OF A FILE

Write a Python program that computes the MD5 sum of the data in a file. You don't need to worry about any of the file's metadata, such as last modified time or even the file's name, only its contents.

You should check your work for Exercise 2.4. If you are using an Ubuntu Linux system, the md5sum utility is already installed. Run this utility from the command line with a file as input and see if it produces the same hex digest as your utility.

checking your program against the system utility

```
$ md5sum myfile.txt
d41d8cd98f00b204e9800998ecf8427e myfile.txt

$ python md5_of_file.py myfile.txt
d41d8cd98f00b204e9800998ecf8427e
```

FOOTNOTE 5

Sometimes called the "MD5 sum", where "sum" is short for "checksum", a name with some interesting (and long) history of its own, from error detection in digital transmissions.

BOOK PAGE 37

12.2 A real example verifying an Ubuntu download

Speaking of Ubuntu, this is a perfect example of using hashes for file integrity. Visit the web site for Ubuntu releases. At the time of this writing, the web site is **https://releases.ubuntu.com**. If you take a look at the "Bionic Beaver" distribution, for example, you will find that there are a number of files available for download. In particular, there are two ISOs, but they are available directly or through other downloading technologies such as BitTorrent.

There is also a file called MD5SUMS. Take a look. For this distribution, the contents of this file should be as follows:

MD5SUMS

```
f430da8fa59d2f5f4262518e3c17246 *ubuntu-18.04.1-desktop-amd64.iso
9b15b331455c0f7cb5dac53bbe050f61 *ubuntu-18.04.1-live-server-amd64.iso
```

Once downloaded, you can verify that the data is uncorrupted by running an MD5 sum on the ISO.

12.3 How is the MD5 hash value helpful?

It will not protect you from somebody that has compromised the Ubuntu web site. If they upload a fake Ubuntu to the web server, they can upload a fake MD5 sum as well.

The MD5 sum does, however, make it easier for you to obtain the Ubuntu ISO from other sources and know that it is authentic. For example, suppose that you were about to download the ISO file directly from the Ubuntu web site when a coworker stops by and says that you can use the already-downloaded one they have on a USB drive. You can download the (relatively small) file of MD5 sums from Ubuntu's official site and check them against the (much larger) files on the drive before trusting them.

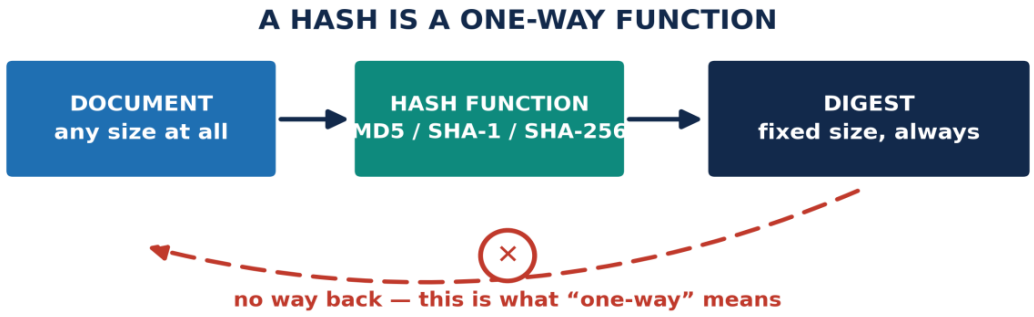


Figure 12.1 File integrity in one picture. The digest is small enough to fetch from a trusted source; the file itself can come from anywhere.

Looking in the Ubuntu directory, you will also see a file called SHA1SUMS and SHA256SUMS. What are these?

12.4 What "broken" really means

So far, we have only talked about MD5 as a way of teaching some of the principles of hashing. MD5 was a standard approach to cryptographic hashing for a long time too, but it has been broken: people have

discovered methods much faster than brute force for inducing collisions, so it is being phased out in favour of other hash functions.

Interestingly enough, being “broken” often means “someone can solve a problem in an order of magnitude less time than brute force”. For example, that might mean that preimage values can be found in 2^{127} tries on average instead of 2^{128}. That is still hard, just not as hard as it should be.

When looking at articles indicating that something has been broken, it is important to find out exactly what that means. Does it mean one of the fundamental properties no longer holds? Does it mean it holds, but isn’t as hard to get around? What if it’s more than one property? These things matter.

With MD5, researchers have found a way to “break” preimage resistance. How much faster? Well, their algorithm takes slightly longer than 2^{123} tries or, in decimal, 10,633,

> **A CAUTION ABOUT SECURITY HEADLINES**
>
> “Algorithm X is broken” in a news headline can mean anything from “a theoretical attack shaved five bits off the security margin” to “your laptop can forge a certificate over lunch”. Those two situations demand completely different responses.
>
> Always ask three questions: which property was broken, how much cheaper did the attack become, and is the attack practical on real hardware today? For MD5 the answers are: collision resistance, catastrophically cheaper, and yes.

BOOK PAGE 38

… 823,966,279,326,983,230,456,482,242,756,608 tries. This attack is considered theoretical because it is still not useful in practice: 2^{123} is still huge.

$$2_{123} = 10,633,823,966,279,326,983,230,456,482,242,756,608$$

13.1 Collision resistance, on the other hand

On the other hand, MD5 has been shown to be very, very broken where collision resistance is concerned. It is reasonably easy to create two inputs that produce the same MD5 output. This has been shown to enable a practical attack for getting a false certificate for use in TLS, which is used for all kinds of secure Internet communication. We won't get into the details here, because we haven't talked about certificates yet, but we will revisit this when we get to TLS at the end of the book.

13.2 Collision resistance is not second-preimage resistance

On the other hand, collision resistance is not the same as second-preimage resistance. Remember that second-preimage resistance prevents you from finding a second member of the preimage for an output when you already have the first. Even though MD5's collision resistance is broken, its preimage resistance is not.

Returning to our Ubuntu distribution example, if you are getting your distribution from an intermediary, they are not able to create an alternate distribution with the same MD5 digest.

13.3 But the Ubuntu organisation itself could

The Ubuntu organization, however, could exploit the MD5 collision resistance weakness to create two separate distributions that have the same MD5 sum. Perhaps, in conjunction with a government, they could sell one distribution with all kinds of tracking software to another government hostile to the first. The MD5 sum could not be used to ensure that the same ISO was being distributed to all parties.

> **WHY THIS DISTINCTION MATTERS SO MUCH**
>
> A collision attack needs the attacker to control BOTH documents. That is useless against a third party distributing somebody else's file, but devastating when the attacker is the one who authored the file in the first place.
>
> This is exactly the shape of the Flame malware attack and of the forged TLS certificate work: the attacker crafts two documents, gets the innocent one signed, then swaps in the malicious one carrying the same digest.

13.4 Suspicion spreads

Also, once a cryptographic algorithm is broken in one way, there is increased suspicion that it is broken in other ways as well. So even though nobody has demonstrated a practical attack on MD5's preimage or second-preimage resistance, many cryptographers worry that such vulnerabilities exist.

13.5 The bottom line on MD5 and SHA-1

To reiterate the warning at the beginning of the chapter, DO NOT USE MD5. It has been deprecated for over 10 years (decimal), and some of its security flaws have been known for two decades.

The SHA-1 hash is another algorithm that was widely viewed as the replacement for MD5. SHA-1's collision resistance has also recently been broken, however, as researchers have shown that it is relatively easy to create two inputs that hash to the same output. So, as with MD5, DO NOT USE SHA-1, either.

At the time of this writing, best practice is to use SHA-256. Fortunately, this means very little to you if you are using hashlib: just change the hasher, as shown in Listing 2-6.

Algorithm	Verdict	Why
MD5	DO NOT USE	Collision resistance thoroughly broken; deprecated for over a decade
SHA-1	DO NOT USE	Collision resistance broken by researchers; being removed everywhere
SHA-256	Use this	No practical attacks; the current best practice at the time of writing

BOOK PAGE 39

14. Listing 2-6 Change to SHA-256

Listing 2-6. Change to SHA-256

```
>>> import hashlib
>>> hashlib.md5(b'alice').hexdigest()
'6384e2b2184bcbf58eccf10ca7a6563c'
>>> hashlib.sha1(b'alice').hexdigest()
'522b276a356bdf39013dfabea2cd43e141ecc9e8'
>>> hashlib.sha256(b'alice').hexdigest()
'2bd806c97f0e00af1a1fc3328fa763a9269723c8db8fac4f93af71db186d6e90'
```

14.1 Line by line

Line(s)	What that line actually does
1	The same import as before. Every algorithm lives behind the same module.
2 – 3	MD5 of `b'alice'` the value you have now seen half a dozen times. 32 hex characters.
4 – 5	SHA-1 of the identical input. Note that *absolutely nothing* changed except the function name. 40 hex characters.
6 – 7	SHA-256, again identical in form. 64 hex characters. This is the whole point of the listing: switching to a stronger hash is a one-word edit.

14.2 Compare the output sizes

You should notice that these different hash algorithms have different lengths. MD5, of course, outputs 16 bytes (128 bits). If it isn't obvious, SHA-1's output is 20 bytes (160 bits). And, more simply, SHA-256's output is 32 bytes (256 bits).

Algorithm	Digest bytes	Digest bits	Hex characters	Ideal preimage cost
MD5	16	128	32	2^{128}
SHA-1	20	160	40	2^{160}
SHA-256	32	256	64	2^{256}

14.3 The numbers written out

If you thought it would take a long time to invert MD5 (find a preimage for a given output), take a look at SHA-1. Because the output is 160 bits, it will take 2^{160} tries, or

1,461,501,637,330,902,918,203,684,832,716,283,019,655,932,542,976

tries to find a preimage. And SHA-256 requires 2^{256} tries, or

115,792,089,237,316,195,423,570,985,008,687,907,853,269,984,665,640,564,039,457,584,007,913,129,639,936.

Good luck with that!

VERIFY EVERY DIGEST IN THIS SECTION YOURSELF

Open your interpreter and type all three lines of Listing 2-6. You must get exactly the values printed above. This is a five-second habit that will save you hours later: when a checksum does not match, the very first thing to establish is whether your tooling is correct before you suspect the file.

BOOK PAGE 39

15. Pass Hashwords…Er…Hash Passwords

Another common use for hash functions is password storage. When you create an account on a web site, for example, they almost never store your password. Typically, they store a hash of the password. That way, if the web site is ever compromised and the password file is stolen, the attacker cannot recover anyone's password.

15.1 What that actually means at login time

What does this mean? When you send your password (over a secure channel, via HTTPS), the server doesn't need to store it to check it. When you registered, your password was hashed, and the hash was stored. We'll call that H(password). When you go to log in later, you send a password that we'll call the "proposal"; you're proposing that this is your true password, and the server needs to verify that.

So, you attempt to log in by sending your proposed password over a secure connection, and the server now has two things for you: it can look up H(password) from your username, and it has the proposal that you just submitted. All it has to do is check that H(proposal) = H(password) and let you through if they are the same.

Stage	What the client sends	What the server stores	What the server computes
Registration	the password, once, over HTTPS	H(password)	H(password), then discards the password
Login	the proposal, over HTTPS	nothing new	H(proposal), then compares
After a breach		attacker gets H(password) only	

WHY THE ORDER OF OPERATIONS MATTERS

The server hashes the proposal and compares digests. It never reverses the stored digest, because it cannot that is the whole point of using a one-way function.

This also explains a familiar experience: a site that can email you your original password is storing it in a recoverable form and has just told you it is unsafe. A site that can only offer you a reset link is doing it correctly.

BOOK PAGE 40

15.2 "Why not hash it in the browser?"

What if you don't trust the service to not actually store your password? That can be a valid worry, particularly since we've seen so many sites with stolen passwords in recent years. Why not use the power of JavaScript to hash your password in the browser, then send that to the server? Then the server never even sees your password in memory, let alone in its database!

There are a few big problems with this:

- The code that hashes the password in your browser came from that server in the first place, so you still have to trust the service.
- If you don't have a secure channel for your password, then someone can read it in transit. If you do have a secure channel, then you might as well just send the password. You have to trust the service already.
- If you send a hash successfully, that just became your password. Yes, you can generate it from some other easy-to-remember thing, but now you have to protect that hash value as well. The server has to hash your hash anyway, so that an attacker that makes off with the database can't just log in using what's stored there.

15.3 The right way to use a hash as a password

In short, if you were to use a hash as your password, the right way is to generate that hash from your password and the site name of interest using a tool separate from your browser, and then use the result as your password. This is essentially the same thing as just creating a brand new password and remembering it in a secure place, like a password manager.

Just do that instead. Then the server will never see a password that you use somewhere else, because you created a brand new random one for it.

15.4 Better than hashing: more factors

Far better than trying to solve security with a hash is to use multiple forms of authentication that are proven to make it harder to steal your identity online, typically involving a hardware token attached to your computer.

BOOK PAGE 41

NOT ALL SECOND FACTORS ARE EQUAL

Most common forms of two-factor authentication don't help and actually make things worse. Secret questions are one of those. It's usually easy to get answers to them, and if it isn't, they're just one more hard thing to remember, unless written down. Plus, now you have several things that can be used as a password for the site, which means attackers have more opportunities to get in by guesswork. SMS has been shown to be critically weak and easy to hijack as well, so having a code sent via SMS to your phone is no good.

Properly deployed challenge-response hardware tokens don't have any of these problems. They are something you possess instead of just another thing you know, and they can't be guessed or spoofed by people listening in on the connection or pretending to be some other site's login form. They can't be given over the phone accidentally, and they can't be forged.

Ultimately you need two or more factors for authentication anyway for true security. "Fixing passwords" is the wrong place to look for a complete solution.

Second factor	Verdict	Reason
Secret questions	Harmful	Answers are often public; effectively extra passwords to guess
SMS one-time codes	No good	Critically weak; SIM-swap and interception hijacks are routine
Authenticator app codes	Better	Not phishing-proof, but far harder to intercept than SMS
Challenge-response hardware token	Best	Something you possess; cannot be guessed, spoofed, phoned or forged

15.5 Looking at a stolen password file

If server-side hashing is correctly used and if an attacker steals the password file, they will see something like this. From looking at it, can you tell smithj's password?

the stolen password file

```
...
smithj,5f4dcc3b5aa765d61d8327deb882cf99
...
```

Look closely. Have you seen that hash value before?

The eagle-eyed reader will remember that hash value from the beginning of the chapter exercises. You were asked to look for that value online. What did you find?

That hash value is the MD5 hash of "password", and yes, that password is still used far too often. But the deeper problem here is that hash values are deterministic: the same input always hashes to the same output. If an attacker has seen the MD5 sum of "password" once, he will be able to look for that same digest in every password file stolen. How can we fix this?

First, let's not assume that we can make people stop using dumb passwords. Let's assume that they're going to and we need to fix it anyway. We'll start with the digest itself.

BOOK PAGE 42

15.6 First attempt: change the algorithm

Recall that MD5 is not broken (in practice) with respect to preimage resistance or second-preimage resistance. So there is no practical attack, at present, for inverting this hash value to the password. Nevertheless, MD5 is broken and should not be used! So let's take a look at a new password file.

with SHA-1 instead

```
...
smithj,5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
...
```

Any idea what smithj's password is now? Yes, it is still "password" but now it's hashed under SHA-1. That's better, right? Oh yeah, SHA-1 is broken and should not be used! Let's try one more time!

with SHA-256 instead

```
...
smithj,5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
...
```

There! Finally! We're using a hash algorithm without known vulnerabilities. That's better, but the problem with deterministic hashing is still a problem. If the attacker knows that this hash maps to the SHA-256 of "password", smithj is still compromised.

> **THE LESSON OF THIS THREE-STEP SEQUENCE**
>
> Upgrading the algorithm fixed a real problem and did absolutely nothing for this one. A stronger hash of a weak password is still a weak stored credential, because the attacker never needed to break the hash they only needed to recognise it.
>
> This is why "we use SHA-256" is not, on its own, an answer to "how do you store passwords?" in an interview.

15.7 The fix: a salt

This is where the idea of a "salt" comes into play. A salt is a publicly known value that is mixed in with the user's password before hashing. By mixing in a salt value, the user's password will not be immediately discernible as it is now.

This salt has to be chosen correctly. It needs to be unique, and it needs to be sufficiently long. One way of doing this is to use os.urandom and base64.b64encode to generate a strong, random salt.

Generating a strong salt

```
1  import os
2  import base64
3
4  salt = base64.b64encode(os.urandom(16))
5  print(salt)     # e.g. b'cei6LtJVQYSM+n6Cty0O2w=='
```

Line(s)	What that line actually does

4	`os.urandom(16)` asks the operating system for 16 bytes from its cryptographically secure random source. This is not `random.random()` that generator is predictable and must never be used for anything security-related.
4	`base64.b64encode` turns those raw bytes into printable ASCII so that the salt can be stored in a plain text file alongside the digest. The `==` at the end is base64 padding, not part of the value.

FOOTNOTE 6

The requirement is uniqueness, not randomness, but randomness provides an approach that works well for our examples.

BOOK PAGE 43

Obviously, your salt output will be different from that shown in the code listing and will be different every time you call it.

16.1 Using the salt

Once you have the salt, you store it, then mix the password and the salt with concatenation. For example, prepend the password with the salt before hashing. Now, if the attacker gets your password file, it will be impossible to "recognize" the password from any kind of pre-computed table.

They can still try hashing the salt plus "password" to see if anything matches, though. Guesswork is always a strategy, and it's a particularly good one for most people's password choices.

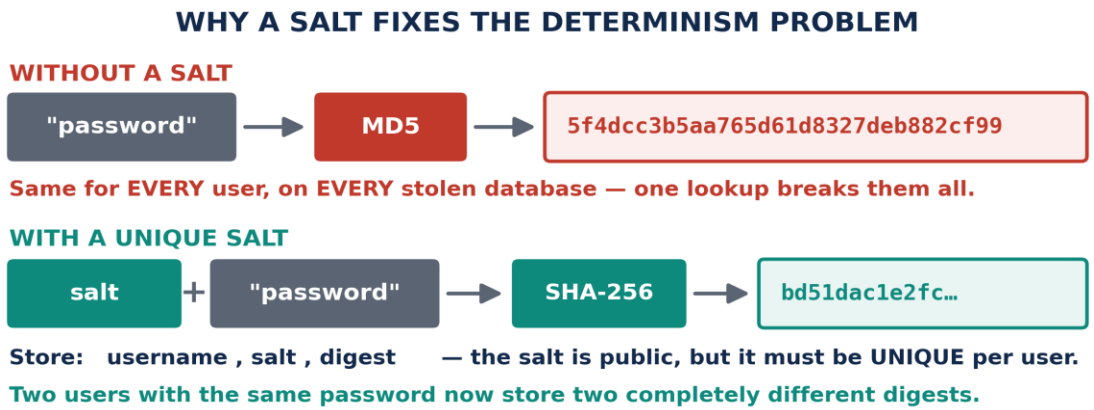


Figure 16.1 The salt does not make one guess harder. It makes the attacker repeat the entire guessing effort separately for every single user.

16.2 Should the same salt be reused?

It should be easy to see that the same salt has to be used every time for checking a user's password. But should the same salt be used for multiple users? Could you generate the salt once for an entire web site and just reuse it?

The answer is a very strong "No!" Can you think of why? At the very least, it means that it is instantly easy to recognize if two users are using the same password. Thus, best practice is to store the user name and salt along with the password hash.

16.3 The corrected password file

If our friend smithj has the terribly chosen password, "password", at least it will be stored correctly on our system:

```
username , salt , digest
...
smithj,cei6LtJVQYSM+n6Cty0O2w==,
 bd51dac1e2fca8456069f38fcce933f1ff30a65320877b596a14a0e05db9567
...
```

Field	Contents	Secret?
Username	smithj	No
Salt	cei6LtJVQYSM+n6Cty0O2w==	No salts are public by design
Digest	bd51dac1e2fc…	No but it must be useless to the attacker
Password	never stored anywhere	Yes and the server never has it

16.4 Doing better still: purpose-built algorithms

We have now walked through the basics of password storage, but there are better algorithms. They are built on the same principle but do additional steps to make it even harder for an attacker to invert the password.

One highly recommended algorithm for password storage is called scrypt by Colin Percival and described in RFC 7914. Other popular ones are the newer bcrypt and the algorithm considered by some to be its successor, Argon2.

Algorithm	Where to find it	Note
scrypt	described in RFC 7914; available in the cryptography module	The book's worked example see Listings 2-7 and 2-8
bcrypt	https://pypi.org/project/bcrypt/	Quite good; has only one "difficulty" parameter, making it easier to use correctly than approaches with many parameters
Argon2	https://pypi.org/project/argon2/	Considered by some to be bcrypt's successor

Fortunately, using scrypt is easy using the cryptography module you set up in Chapter 1. Listing 2-7 is an example derived from the cryptography module's online documentation. The listing derives the key (hash) to be stored on the file system.

FOOTNOTE 7

The bcrypt algorithm is quite good and has only one "difficulty" parameter, making it easier to use correctly than approaches with many parameters.

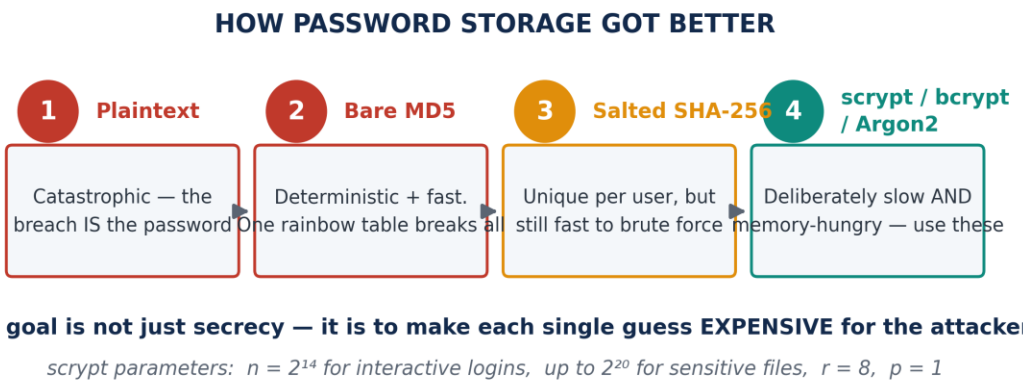


Figure 16.2 The four stages of password storage. Most real breaches you read about were still stuck at stage 1 or 2.

BOOK PAGE 44

17. Listings 2-7 and 2-8 scrypt

Listing 2-7. Scrypt Generate

```
1  import os
2  from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
3  from cryptography.hazmat.backends import default_backend
4
5  salt = os.urandom(16)
6
7  kdf = Scrypt(salt=salt, length=32,
8         n=2**14, r=8, p=1,
9         backend=default_backend())
10
11 key = kdf.derive(b"my great password")
```

17.1 Line by line

Line(s)	What that line actually does
1	We need `os.urandom` for the salt.
2	Imports the Scrypt key-derivation function. Note the word **`hazmat`** in the path it stands for "hazardous materials", and it is the module authors telling you that these are low-level primitives that can be misused. That naming is itself a piece of security design.
3	Imports the default backend, which is the underlying engine (usually OpenSSL) that performs the actual computation.
5	Generates a fresh 16-byte random salt one per user, exactly as Section 16 required.
7	Constructs the KDF. `length=32` asks for a 32-byte (256-bit) output key.
8	The three tuning parameters. `n` is the CPU/memory cost, `r` the block size, `p` the parallelisation factor. Section 18 explains how to choose them.
9	Supplies the backend to the constructor.
11	Runs the derivation. This deliberately takes a noticeable fraction of a second that slowness **is the security feature**, not a performance bug.

Both the key and the salt must be stored to disk. The scrypt parameters must be fixed or must also be stored. We will walk through these parameters in a moment, but first, verification is depicted in Listing 2-8 (it is presumed that salt and key are restored from disk).

Listing 2-8. Scrypt Verify

```
1  kdf = Scrypt(salt=salt, length=32,
2         n=2**14, r=8, p=1,
3         backend=default_backend())
4  kdf.verify(b"my great password", key)
5  print("Success! (Exception if mismatch)")
```

Line(s)	What that line actually does
1 – 3	Rebuilds the KDF with **exactly the same salt and parameters**. Change any one of them and verification fails, which is why they must be stored alongside the key.
4	`verify()` re-derives the key from the supplied password and compares. Note what it does *not* do: it does not return True or False. On mismatch it raises an exception.
5	Reached only on success. The comment in the book makes the control flow explicit an exception is the failure path.

WHY verify() RAISES INSTEAD OF RETURNING FALSE

A returned boolean is easy to ignore. Real code has shipped with bugs like if check_password(p): pass where the result was silently discarded and everyone got in.

An exception cannot be ignored by accident the program stops unless you deliberately catch it. The library is also performing a constant-time comparison internally, which protects against timing attacks. Never write your own == comparison on digests.

18. Pick Perfect Parameters

With regard to the scrypt parameters, let's talk about backend first. The cryptography module is primarily a wrapper around a lower-level engine. For example, the module can make use of OpenSSL as such an engine. This makes the system faster (because computations aren't being done in Python) and more secure (because it relies on a robust, well-tested library). Throughout this book, we will always rely on default_backend().

The other parameters are specific to scrypt. The length parameter is how long the key will be once the process is finished. In these examples, the password is processed into an output of 32 bytes. The parameters r, n, and p are tuning parameters that…

BOOK PAGE 45

… impact how long it will take to compute and how much memory is required. To better protect your password, you want the process to take longer and require more memory, preventing attackers from compromising large chunks of a database at once (every compromise should take a long time).

18.1 Recommended values

Fortunately for you, recommended parameters are available. The r parameter should be 8, and the p parameter should be 1. The n parameter can vary based on whether you are doing something like a web site that needs to give a relatively prompt response or something more securely stored that does not need quick responsiveness. Either way, it must be a power of 2. For the interactive logins, 2^{14} is recommended. For the more sensitive files, a number as high as 2^{20} is better.

Parameter	Recommended value	What it controls
r	8	Block size leave it alone
p	1	Parallelisation leave it alone
n	2^{14} for interactive logins	CPU and memory cost. Must be a power of 2.
n	up to 2^{20} for sensitive files	64× more work than 2^{14}, for data that can afford the wait
length	32	Size of the derived key, in bytes

18.2 The general lesson about parameters

This is actually an excellent segue into a more general discussion about parameters. A lot of the security in cryptography depends on how parameters are set. Unless you are a cryptography expert, know the exact details of the algorithm, and understand why they are what they are, it may be difficult to choose them properly.

It is important that you familiarise yourself with what the parameters mean, at least at a high level, and how they should be used in different contexts. Refer to trusted sources, such as **https://cryptodoneright.org**, for advice and recommendations. Keep an eye on these sources too. What is presumed to be secure can change as new attacks and computational resources are unveiled.

> **THIS IS THE CHAPTER 1 LESSON RETURNING**
>
> Chapter 1 said correct cryptographic usage is heavily dependent on context, and that a big part of learning cryptography is learning how parameter settings change the operation. The scrypt n parameter is that abstract warning made completely concrete: the same algorithm, the same code, and a single exponent separating a well-protected database from an easily harvested one.

BOOK PAGE 45 – 46

19. Cracking Weak Passwords

Let's take a look at how attackers try to crack passwords. Unfortunately for smithj, choosing such a bad password means that he will most likely be compromised if the password file gets stolen, since attackers will try common words (including words in other stolen databases) against all the hashes anyway. But even less sophisticated methods would probably figure out the password as well.

In this section, we are going to practice cracking weak passwords using the least sophisticated method of all: brute force. This exercise is meant to reinforce why good passwords are so important.

The scenario is this: an attacker has a password file with usernames, salts, and password hashes. What can they do? Well, they could just try all lowercase letter combinations up to a certain length, starting, for example, with "a", "b", "c", and so on.

19.1 Listing 2-9 Alphabet Permutations

To make these exercises a little bit easier to start, Listing 2-9 shows some simple code for generating all possible combinations of an alphabet set up to a maximum length.

Listing 2-9. Alphabet Permutations

```
1  def generate(alphabet, max_len):
2    if max_len <= 0: return
3    for c in alphabet:
4      yield c
5    for c in alphabet:
6      for next in generate(alphabet, max_len-1):
7        yield c + next
```

19.2 Line by line

Line(s)	What that line actually does
1	Two parameters: the set of characters to draw from, and the longest string to produce.
2	The recursion's base case. When the remaining length reaches zero we stop. A bare `return` inside a generator ends iteration rather than returning a value.
3 – 4	Emits every single-character string first. **`yield`** is what makes this a **generator**: the function hands back one value and pauses, resuming where it left off on the next request. Nothing is ever stored in a list.
5 – 7	The recursive step. For each character c, generate every string of length up to max_len−1 and prepend c to it. This produces all strings of length 2 up to max_len.
6	A style note: `next` shadows a Python built-in. The book uses it for readability; in your own code prefer `suffix` or `rest`.

WHY A GENERATOR AND NOT A LIST

Line(s)	What that line actually does

Building a list of every 6-character lowercase string would need over 300 million entries and gigabytes of RAM. A generator produces them one at a time and keeps only the current recursion stack in memory, so this seven-line function can enumerate a space your machine could never store.

This is the single most useful Python idiom in the whole chapter, and it applies far beyond cryptography log processing, data pipelines, and anywhere the data is bigger than memory.

19.3 Trying it out

generate('ab', 2)

```
>>> list(generate('ab', 2))
['a', 'b', 'aa', 'ab', 'ba', 'bb']
```

Calling generate('ab', 2) will generate 'a', 'b', 'aa', 'ab', 'ba', 'bb'. Using helpful sets in the built-in string module, such as

- string.ascii_lowercase
- string.ascii_uppercase
- string.ascii_letters

makes the following exercises fairly easy. Recall that hashing algorithms require bytes as inputs, so don't forget to do an encode operation before passing these generated strings to the hashing function, like this:

```
1 string.ascii_letters.encode('utf-8')
```

ASCII letters encode correctly to bytes, so this will not lead to incorrect hashing or unexpected behaviors.

EXERCISE 2.5 THE POWER OF ONE

Write a program that does the following ten times (so, ten full loops with the time computed):

• Randomly select a single, lowercase letter. This is the "preimage seed."

• Use MD5 to compute the hash of this initial letter. This is the "test hash."

• In a loop, iterate through all possible lowercase one-letter inputs.

– Hash each letter in the same way as before, and compare against the test hash.

– When you find a match, stop.

• Compute the amount of time it took to find a match.

How long, on average, did it take to find a match for a random preimage seed?

BOOK PAGE 47

19.4 A skeleton for Exercise 2.5

A starting point finish it yourself before reading further

```
1  import hashlib
2  import random
3  import string
4  import time
5
6
7  def crack_one_letter():
8    seed = random.choice(string.ascii_lowercase)
9    test_hash = hashlib.md5(seed.encode('utf-8')).hexdigest()
10
11   start = time.time()
12   for candidate in string.ascii_lowercase:
13     guess = hashlib.md5(candidate.encode('utf-8')).hexdigest()
14     if guess == test_hash:
15       return candidate, time.time() - start
16   return None, time.time() - start
17
18
19 if __name__ == "__main__":
20   total = 0.0
21   for trial in range(10):
22     found, elapsed = crack_one_letter()
23     print("trial {:2}: found '{}' in {:.6f} s".format(
24       trial + 1, found, elapsed))
25     total += elapsed
26   print("average: {:.6f} s".format(total / 10))
```

Line(s)	What that line actually does
8	Picks the secret at random this stands in for the unknown password.
9	Computes the target digest. This is all the attacker is assumed to have.
11	Starts the clock *after* the setup, so we time the attack and not the preparation.
12 – 14	The attack itself: hash every candidate and compare. Note that we compare digests, never the secret the attacker has no access to it.
15	Returns as soon as a match is found, which is what “when you find a match, stop” asks for.
21 – 26	Runs ten full trials and reports the average, as the exercise requires.

EXERCISE 2.6 THE POWER OF ONE, BUT BIGGER!

Repeat the previous exercise, but use an increasingly bigger input alphabet set. Try the test with both lowercase and uppercase letters. Then try it with lowercase letters, uppercase letters, and numbers. Finally, try all printable characters (string.printable).

- How many total symbols are in each input set?
- How much longer does each run take?

EXERCISE 2.7 PASSWORD LENGTH'S EFFECTS ON ATTACK TIME

Repeat the previous exercise, but this time for two-symbol inputs. Then try it with three and four symbols at a time. How much longer does it take to invert the randomly chosen input?

19.5 The mathematics behind those two exercises

You will notice that increasing the length of the password and increasing the size of the alphabet both increase the time it takes to invert the hash. Let's look at the math.

When using just lowercase letters, how many possible one-symbol inputs are there? Rather trivially, there are 26 lowercase letters in ASCII, so 26 one-symbol inputs. At worst, it will take 26 hash computations to invert a one-letter password.

But, if we have both lowercase and uppercase letters, this increases the number of hashes needed to 52. Adding digits increases this to 62. There are 100 characters in string.printable. That's nearly a fourfold increase of the worst-case number of hashes required to do brute-force inversion.

What about when we increase the size to two input symbols? How many two-symbol passwords are there using just lowercase letters? If you can have 26 characters for the first symbol and 26 characters for the second symbol, then there are 26 * 26 = 676 total combinations. That's quite a jump!

BOOK PAGE 48

Now look what happens if you use two symbols drawn from the 52 uppercase and lowercase letters. The math works out to be 52 * 52 = 2704! Doubling the size of the input set quadrupled the complexity for two-symbol inputs! If we throw in digits, the worst-case computation is 3844 hashes, and for all printable ASCII characters, it is around 10,000 hashes.

Alphabet	Size	1 symbol	2 symbols	3 symbols	4 symbols
lowercase	26	26	676	17,576	456,976
+ uppercase	52	52	2,704	140,608	7,311,616
+ digits	62	62	3,844	238,328	14,776,336
string.printable	100	100	10,000	1,000,000	100,000,000

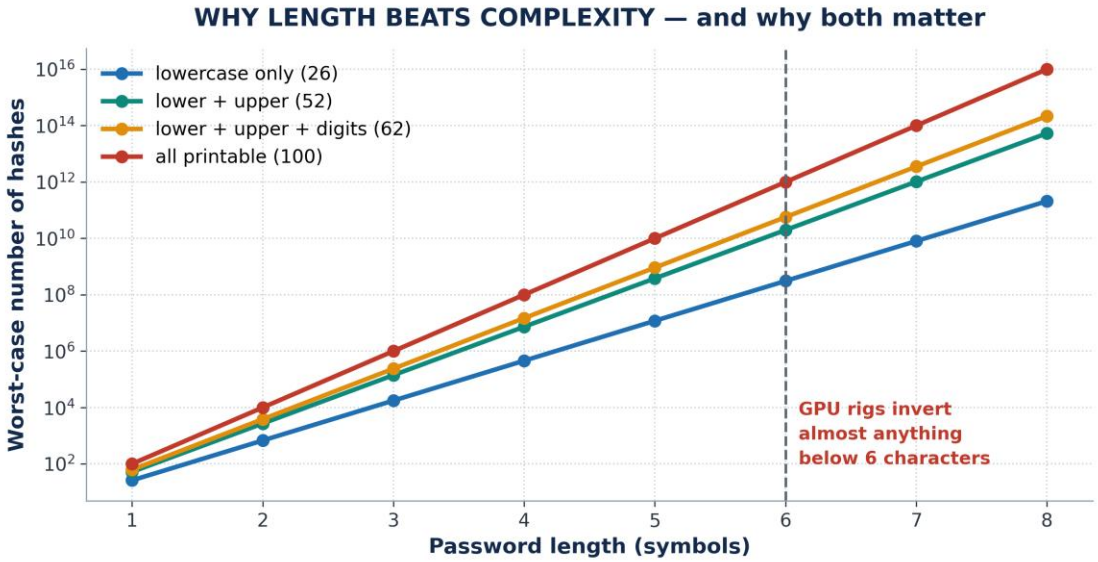


Figure 19.1 The vertical axis is logarithmic, so each gridline is a factor of ten. Every extra character multiplies the work; every extra character class multiplies it again.

Do the math for three, four, and five symbols, and you can easily see why longer passwords matter. Hackers with GPU-enabled rigs are able to invert anything smaller than six characters, and most passwords under eight, so at a minimum, passwords should be that long. And for the reasons demonstrated here, choosing from all printable letters greatly increases the complexity.

EXERCISE 2.8 MORE HASH, MORE TIME

Choosing a complex-to-invert password is the responsibility of the user, but the systems storing the passwords can also slow down attackers by using a more complicated hashing function. Repeat any of the preceding exercises that use MD5, but now use SHA-1 and SHA-256 instead. Record how much longer it takes to get through the brute-force operations. Finally, try out brute force using scrypt. You might not get very far!

WHY YOU WILL NOT GET VERY FAR WITH scrypt

MD5 runs in microseconds, so a GPU can compute billions per second. scrypt with $n = 2^{14}$ is deliberately engineered to take a noticeable fraction of a second and to demand a substantial block of memory for every single attempt.

That memory requirement is the clever part: GPUs get their speed from thousands of tiny cores with very little memory each, so a memory-hard function removes the attacker's hardware advantage rather than merely slowing them down. This is exactly what Section 17 meant by "the slowness is the security feature".

19.6 One final note on password strength

Just because a password is big doesn't mean it is secure. Attackers will also use large dictionaries to look for known words and phrases, even with various common number or symbol substitutions. A password such as "chocolatecake" is pretty long, but still easily broken. Randomly chosen letters or words are still the best bet. The key is that they are "random," meaning you would never find them in any real writing or common transformations on real writing. Typically, choosing passphrases that are composed of common utterances reduces a successful attack to seconds instead of years.

BOOK PAGE 48 – 49

20. Proof of Work

Another area where hashing is used extensively is so-called “proof-of-work” schemes in blockchain technologies. To introduce this, we need to do a very quick overview of how blockchains work.

20.1 A blockchain is a distributed ledger

The basic idea of a blockchain is a “distributed ledger.” The system is a ledger because it records information related to transactions between participants. It can also store additional information, but the primary operations are transactions. It is a distributed ledger because its contents are stored across the set of participants and not in any central location.

The problem is that there is no central location to enforce the correctness of the system. How does the ledger not get corrupted (intentionally or otherwise) by the users? Note that we won’t go into the ledger in detail here, but we do want to talk about the blocks that a ledger is composed of.

20.2 Blocks

Every transaction must be stored in a block. There’s nothing special about a “block”; it’s just a collection of data. Each transaction within the block must be digitally signed by the transactor (we will discuss signatures more in Chapter 5, but for now, simply accept that it means nobody can create a transaction for somebody else without their private key).

The overall block structure is protected by a hash. Blocks are copied to the entire set of participants; should anyone try to “lie” about the contents of a block, the data wouldn’t verify correctly and their information would be rejected.

20.3 Mining and the puzzle

How does a new block get created, and how does it get the protective hash? For this part of the discussion, we will use the Bitcoin network blockchain to walk through these concepts. The designer (or designers, the source is actually unknown) of Bitcoin, who goes by “Satoshi Nakamoto,” wanted to control how quickly new blocks could be created and also wanted the system to incentivize participation. The solution was to award bitcoins to the “miner” that produced a new block while making the production of the new block very difficult.

Basically, at any given time, various parties known as miners are searching for the next block in the blockchain. Any user of blockchain can request a transaction. They broadcast their desired transaction throughout the blockchain network and miners will pick them up. The miners take some set of requested transactions (there is a limited number per block) and create a candidate block. This candidate block has all the right pieces of information. It has the transactions, the metadata, and so forth. But it isn’t the next block in the blockchain until the miner can solve a cryptographic puzzle.

20.4 The puzzle itself

That puzzle is to find a special kind of SHA-256 hash value, specifically a value smaller than a certain threshold. As we discussed earlier, finding an input that produces one particular output would take a really, really long time, but finding any output less than a certain value takes a lot less time. Making that threshold smaller reduces the number of valid hashes, requiring more work to find a suitable value, and that's how Bitcoin adjusts the difficulty to account for faster hardware or larger computational pools as time goes on. Ultimately, it takes about 10 minutes for the entire Bitcoin network to find a suitable hash.

BOOK PAGE 50

If it takes less than that on average over a period of weeks, the maximum allowed hash value is decreased.

20.5 Figure 2-1 two blocks, two nonces

Figure 2-1 shows two different example blocks, one with a suitable nonce (a random value that miners are trying to find to produce an acceptable hash) and one without, where the maximum allowed hash value is $2^{236}-1$ (20 leading zeros required). For Bitcoin, the very easiest that a problem is allowed to be is determined by a maximum value of $2^{224}-1$, which would take our little program an average of 2^{12} times longer than before. That translates to 11.3 hours, and the difficulty is much harder than that today.

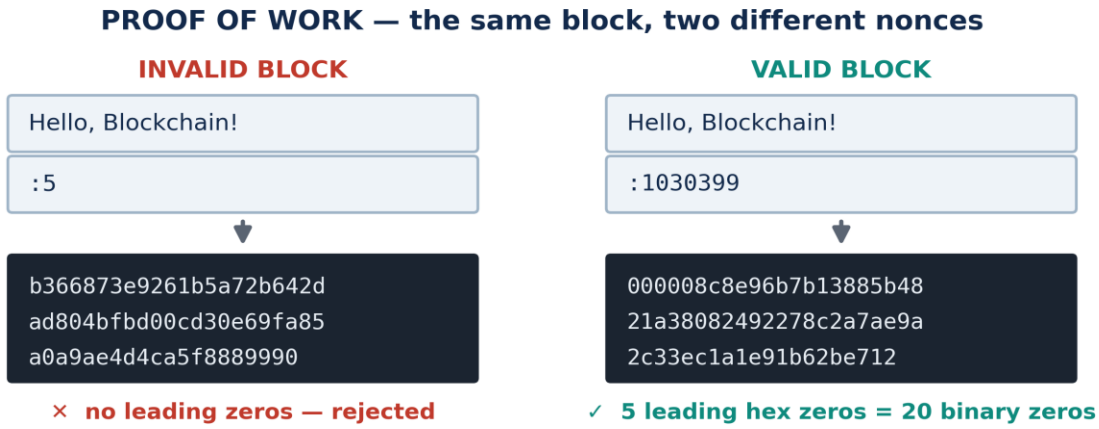


Figure 20.1 Two block hashes with the same content but different nonce values. A nonce that produces a hash with 20 leading binary zeros (5 leading zeros in hexadecimal) is valid. Requiring 20 leading zeros is the same as requiring that the hash number be less than 2^{236}.

Our program definitely won't be beating the network's 10-minute average expectation anytime soon.

20.6 Leading zeros are just a threshold

Saying that the first few bits must be zero is the same, by the way, as saying the hash value number (the hash is just a number, just like any other string of bits) should be less than some threshold that happens to be a power of 2. Since good hash functions (like SHA-256) produce essentially random hash values, the more structure you impose on the hash, the longer it takes to find one that fits.

You can get some intuition for this by thinking about the number of zeros as defining the size of the search space: if you must have a single leading zero, then it's basically a coin flip; it should only take two tries on average to find a suitable hash that starts with a zero bit. If, on the other hand, you need to find a hash with 8 leading zeros, that's a harder problem: 256 different numbers can be represented in 8 bits, so on average it will take 256 attempts to find a suitable value.

Leading zero bits required	Fraction of hashes that qualify	Average attempts needed
1	1 in 2	2
8	1 in 256	256

Leading zero bits required	Fraction of hashes that qualify	Average attempts needed
20	1 in 1,048,576	about a million
32 (Bitcoin's easiest ever)	1 in 4,294,967,296	billions

That's why this strategy is called "Proof of Work"; if you found a suitable hash under the threshold, you had to have done some work (or you broke the hash function, which is deemed to be extremely unlikely, but potentially awesome for you).

BOOK PAGE 51

20.7 Who decides the difficulty?

This raises an interesting question: how do each of the network participants decide how hard the problem should be? It isn't like there is a central authority telling everyone that the difficulty just went from 11 to 12, for example. That would defeat the whole purpose of the network. The "authority" in the network is tacit agreement between participants to use the same algorithms for determining these things. When there is someone on the network doing things differently, their blocks are simply rejected by everyone else and they thus have no incentive to do it wrong. Majority rules.

In the specific case of hashing difficulty, each participant knows the standard algorithm for computing what the number of leading zeros should be and uses that to do mining (or to reject bad proposals from wayward participants looking to compute an easy hash).

20.8 The nonce

You might be asking, however, how a different value of hash is computed when the input data really doesn't change. That's a great question, since hashes are deterministic: they always produce the same output given the same input (they wouldn't be very useful, otherwise!).

The answer is that they change one little piece of the input, called the "nonce." It's just a number, and it isn't part of the actual block data: its sole purpose is to enable the proof-of-work concept. When searching for a suitable hash, the participant tries hashing the block with different values for the nonce, typically searching randomly or merely adding 1 to the last value at each attempt. Eventually a suitable hash value is found and the block is sent to all other participants for validation.

Every participant then verifies the block by performing the hash for themselves, checking the leading zeros against their algorithm, and making sure that their answer matches the submitted hash value. If it's good, they accept it and the chain lengthens.

> **THE ASYMMETRY THAT MAKES THE WHOLE SYSTEM WORK**
>
> Finding a valid nonce takes the entire Bitcoin network about ten minutes. Checking that somebody else's nonce is valid takes one SHA-256 computation microseconds.
>
> Hard to produce, trivial to verify. That asymmetry is the same property behind digital signatures in Chapter 5 and behind the whole of public-key cryptography in Chapter 4. Once you see it here, you will keep recognising it for the rest of the book.

> **EXERCISE 2.9 PROOF OF WORK**
>
> Write a program that feeds a counter into SHA-256, taking the output hash and converting it to an integer (we did this earlier before converting to binary). Have the program repeat until it finds a hash

that is less than a target number. The target number should start out pretty big, like 2^{255}. To make this more like blockchain, include some arbitrary bytes to be combined with the counter.

A skeleton for Exercise 2.9

proof_of_work.py

```
1  import hashlib
2  import time
3
4
5  def mine(block_data, target):
6    nonce = 0
7    start = time.time()
8    while True:
9      candidate = block_data + str(nonce).encode('utf-8')
10     digest = hashlib.sha256(candidate).digest()
11     value = int.from_bytes(digest, byteorder='big')
12     if value < target:
13       return nonce, digest.hex(), time.time() - start
14     nonce += 1
15
16
17 if __name__ == "__main__":
18   data = b"Hello, Blockchain!:"
19   for zero_bits in (8, 12, 16, 20):
20     target = 2 ** (256 - zero_bits)
21     nonce, digest, elapsed = mine(data, target)
22     print("{:2} zero bits -> nonce {:>10} {} ({:.3f} s)".format(
23       zero_bits, nonce, digest[:24], elapsed))
```

Line(s)	What that line actually does
6	The nonce starts at zero and is simply incremented exactly the “merely adding 1 to the last value” strategy described on page 51.
9	Combines the fixed block data with the current nonce. This is why the input changes even though the transactions do not.
10	`.digest()` gives raw bytes rather than hex, because we are about to do arithmetic on them.
11	`int.from_bytes` turns the 32-byte digest into a single enormous integer. This is Section 7’s point made operational: a digest really is just a number.
12 – 13	The threshold test. Requiring k leading zero bits is exactly the same as requiring the value to be below 2 to the power (256 − k).
20 – 21	Runs the search at four difficulties. Each extra four bits multiplies the expected work by 16, so you will watch the timing grow geometrically which is precisely the lesson.

BOOK PAGE 52

21. Time to Rehash

We have covered a lot of information about what hashes are and how to use them, including why you should never use MD5 unless you are teaching people that it is broken, and how to use them for more secure password storage and even crypto currency. Hashing is a powerful and important part of cryptography, and we will be seeing it again and again as we move forward.

Now that we have learned a bit about how to digest a document into a safely representative value, it's time to back up a bit and revisit encryption.

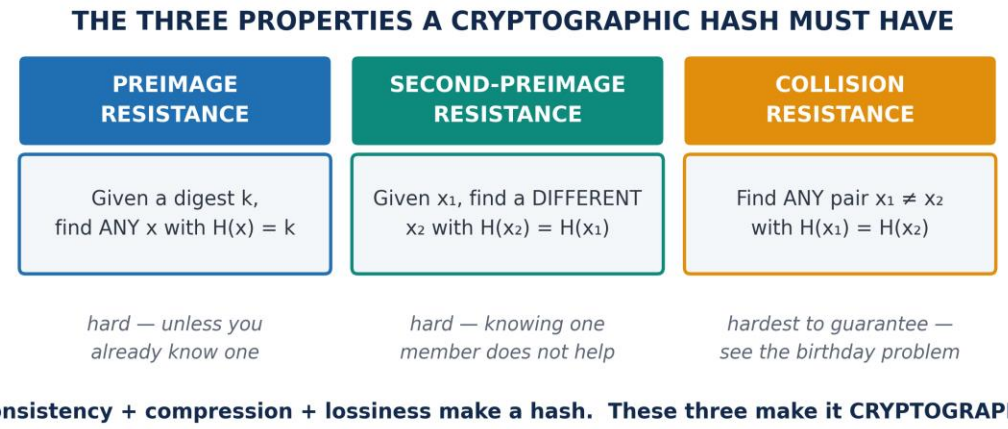


Figure 21.1 Everything in this chapter rests on these three properties. Chapter 5 will build message authentication codes and digital signatures directly on top of them.

Appendix A Chapter Summary at a Glance

Section (book pages)	The one thing to remember
Hashing (21)	A one-way function: any input, fixed-size digest, no way back.
Hash Liberally with hashlib (21–23)	hashlib.md5(b'alice').hexdigest() construct, feed, render.
WARNING: MD5 Is No Good (21)	Used here only because it is weak enough to break in a lab.
Confidentiality vs integrity (22)	Chapter 1 gave secrecy; hashing gives tamper-detection.
Listings 2-1 to 2-3 (22–23)	Empty digest is still 32 hex characters. Output size never changes.
Bytes not strings (24)	Always use b'' or .encode('utf-8'). Python refuses to guess.
Exercise 2.1 (24)	Deterministic, avalanche, fixed length the three lessons.
Listing 2-4 (25)	update() lets you hash a 4 GB file with 32 bytes of state.
Exercise 2.2 (25)	Google is a rainbow table. Determinism is the weakness.
Making a Hash of Education (25–26)	Consistency + compression + lossiness = a hash.
Preimage resistance (27–28)	Given k, finding any x with H(x)=k must be infeasible.
Byte into Non-negative Integers (29–31)	Every four bits is one hex digit. A digest is just a number.
How Hard a Hash! (32)	$2^{128} \approx 3.4 \times 10^{38}$; brute force outlasts the sun.
Second-preimage & collision (33)	Collisions are far cheaper the birthday bound, $2^{n/2}$.
MD5 broken (36–38)	Collision resistance gone; preimage weakened to $\approx 2^{123}$.
Digestible Hash (36–37)	Open files in "rb"; verify downloads against MD5SUMS.
Listing 2-6 (39)	Switching to SHA-256 is a one-word edit.
Pass Hashwords (39–41)	Store H(password); compare H(proposal). Never store the password.
Salts (42–43)	Unique per user, public, stored alongside the digest.
scrypt (44–45)	$n = 2^{14}$ interactive, r = 8, p = 1. The slowness is the feature.
Cracking Weak Passwords (45–48)	Length beats complexity, but you need both. 6 characters is already gone.
Proof of Work (48–51)	Find a nonce so the SHA-256 value falls below a threshold.
Time to Rehash (52)	Hashing recurs throughout the rest of the book.

Appendix B Glossary of Chapter 2 Terms

Term	Meaning
Hash / digest / fingerprint	The fixed-size output produced from an input of any size.
One-way function	Easy to compute forwards, infeasible to reverse.
hashlib	Python's standard-library module providing MD5, SHA-1, SHA-256 and more.
hexdigest()	The digest rendered as hexadecimal text, two characters per byte.
digest()	The digest as raw bytes use this when you need to do arithmetic.
update()	Feeds another chunk of data into an existing hasher.
Consistency	The same input always produces the same output.
Compression	A large input space is mapped onto a small fixed-size output.
Lossiness	Information is destroyed, so the input cannot be recovered.
Domain / range	The set of possible inputs / the set of possible outputs.
Preimage	The set of all inputs x for which H(x) equals a given value k.
Preimage resistance	Hard to find any x with H(x) = k, unless you already know one.
Second-preimage resistance	Given x_1, hard to find a different x_2 with the same digest.
Collision resistance	Hard to find any pair $x_1 \neq x_2$ with the same digest.
Birthday bound	Collisions cost about $2^{n/2}$ work for an n-bit digest, not 2^n.
Avalanche effect	One input bit changing flips about half the output bits.
Brute-force attack	Trying every possible input until one matches.
Rainbow table	A pre-computed lookup table of digests for common inputs.
Salt	A unique, public value mixed into a password before hashing.
Key derivation function	A deliberately slow, memory-hard hash for password storage.
scrypt / bcrypt / Argon2	Recommended key derivation functions for password storage.
Nonce	A number varied to change a hash input without changing the data.
Proof of work	Demonstrating effort by finding a digest below a threshold.
Parity bit	A one-bit non-cryptographic hash used for error detection.

Appendix C Self-Assessment and Viva Questions

C.1 Short-answer questions

1. State the two properties every hash function must have, and the three extra properties a CRYPTOGRAPHIC hash needs.
2. Why does hashing the empty input still produce 32 hex characters?
3. Why does Python raise a TypeError if you pass a plain string to hashlib.md5()?
4. Explain the difference between second-preimage resistance and collision resistance in one sentence each.
5. MD5 produces 128-bit digests. Why do we quote 2^{64} rather than 2^{128} as the collision cost?
6. Somebody says “MD5 is not preimage-resistant, because I Googled a hash and found the password.” What is wrong with that statement?
7. Convert 11101101 to hexadecimal and to decimal, showing your working.
8. Why must the salt be stored in the password file, and why is it safe to make it public?
9. Why is it a bad idea to reuse a single site-wide salt for every user?
10. Why does scrypt.verify() raise an exception instead of returning False?
11. What does the scrypt n parameter control, and what values are recommended for interactive logins and for sensitive files?
12. Explain the role of the nonce in proof of work, and why verification is so much cheaper than mining.

C.2 Practical (laboratory) tasks

1. Complete Exercise 2.1 and verify all eight digests against the table in Section 3.
2. Complete Exercise 2.2 and write one paragraph on what rainbow tables are and why salting defeats them.
3. Run the avalanche measurement in Section 10 on five pairs of similar inputs and tabulate the percentage of bits that differ.
4. Complete Exercise 2.4 and check your program against md5sum on at least one binary and one text file.
5. Reproduce Listing 2-6 and record the digest lengths of MD5, SHA-1 and SHA-256.
6. Implement salted SHA-256 password storage, then demonstrate that two users with the same password store different digests.
7. Run Listings 2-7 and 2-8, then time the derivation at n = 2^{12}, 2^{14} and 2^{16}.
8. Complete Exercises 2.5, 2.6 and 2.7, and plot your measured timings against the theoretical table in Section 19.
9. Complete Exercise 2.8 and record how much slower SHA-256 and scrypt make the same attack.
10. Complete Exercise 2.9 and plot mining time against the number of required leading zero bits.

C.3 Discussion questions

- A teammate proposes hashing passwords in JavaScript before sending them. Give the three counter-arguments from page 40.
- Your college portal emails you your original password after a “forgot password” request. What does that tell you, and what should you do?
- A vendor advertises “military-grade SHA-512 password security” with no mention of salting or of a key derivation function. What questions would you ask?

Appendix D Common Errors and How to Fix Them

Symptom	Likely cause	Fix
TypeError: Unicode-objects must be encoded before hashing	A plain str was passed to a hash function.	Use a b'' literal, or call .encode('utf-8').
AttributeError: 'NoneType' object has no attribute 'hexdigest'	You wrote h = h.update(...). update() returns None.	Call h.update(...) on its own line.
The digest of a file differs from md5sum	The file was opened in text mode.	Open with "rb", never plain "r".
Digest changes between runs on the same file	You are hashing metadata, or reading with decoding.	Hash only the file contents, read as raw bytes.
Hashing a 4 GB file exhausts memory	The whole file was read into one variable.	Read in fixed-size blocks and call update() on each.
scrypt verification always fails	Salt or parameters were not stored, so they differ at verify time.	Store salt, n, r, p and length alongside the key.
scrypt feels far too slow in a web login	n is set too high for interactive use.	Use n = 2^{14} for logins; reserve 2^{20} for sensitive files.
Two users with the same password have the same digest	A single site-wide salt is being reused.	Generate a fresh os.urandom(16) salt per user.
ValueError from int.from_bytes in the mining script	hexdigest() was used where digest() was needed.	Pass raw bytes to int.from_bytes, not hex text.
Brute-force script exhausts RAM	The permutation generator was wrapped in list().	Iterate over the generator directly; never materialise it.